PagedAttention

Optimizing LLL Inference (serving) Systems (09/2023)

Tran Manh Dat

21/02/2025

PagedAttention

1. Problems with Large Language Model (LLM) Inference System
2. PagedAttention - vLLM
3. Related works

**** We Keep Focusing On Serving Model, Not Training Phase***

PagedAttention

1. **Problems with Large Language Model (LLM) Inference System**
2. PagedAttention - vLLM
3. Related works

**** We Keep Focusing On Serving Model, Not Training Phase***

1. Problem

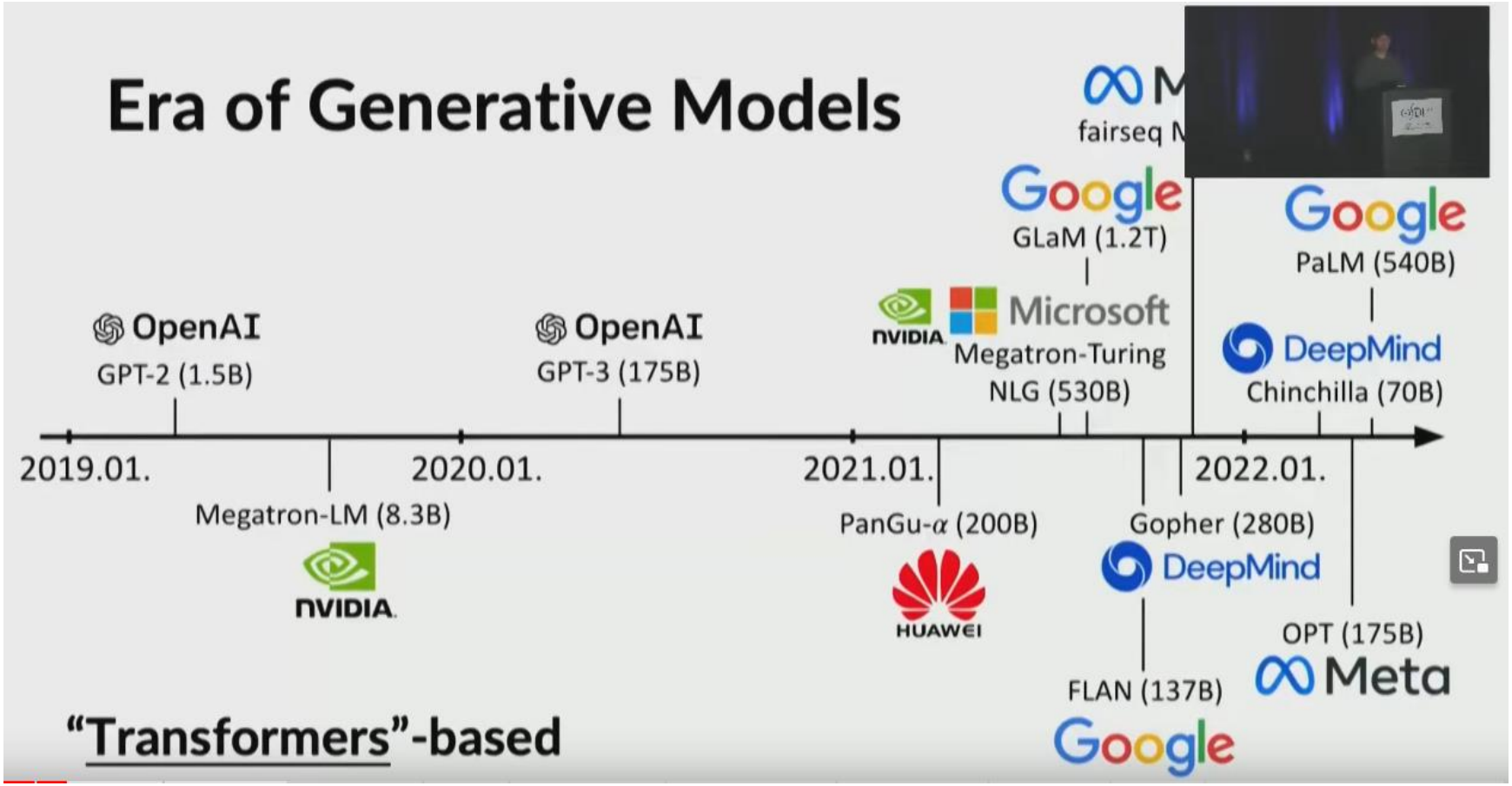


Image from **Orca: A Distributed Serving System for Transformer-Based Generative Models Present (OSDI 2022)**

1. Problem

Serving LLMs is expensive

- **LLMs run on high-end GPUs such as NVIDIA A100**
- **Each GPU can only serve a handful of requests per second.**
  - For LLaMA-13B and moderate-size inputs, 1 A100 can process < 1 requests per second
- **A ton of GPUs are required for production-scale LLM services**

1. Problem

Cost in 2022, Still the same in 2024

→ Increase Throughout ~ reduce cost

- In Azure, a GPT3 175B instance requires 2 VMS, each of which has 8 NVIDIA A100 40GB GPUs

- At Azure East US, the VM price is $27.197/hour.

  2 VMs = $54.394/hour → $39,707.62/month → $476,491.44/year

- If we need to host 400 GPT3 175B instances, $190.6M/year

If perform memory better, for example the increase is 20%, so the cost will reduce: $190 - $190/1.2 = $31.7 (M/year)

1. Problem

To increase Throughout:

- Batching sufficiently (**Orca: A Distributed Serving System for Transformer-Based Generative Models** – 2022 – Closed source FriendliAI)
- Quantization model (float32 → float 16, float 32 → int8): Less memory storage, compute faster.
- Enhance GPU Memory Usage (PagedAttention)

1. Problem

Background:

- Beam Search
- Batching Techniques
- Transformer Structure (Self-Attention)
- LLL Service & Autoregressive Generation (KV Cache)

1. Problem - Background

Beam Search

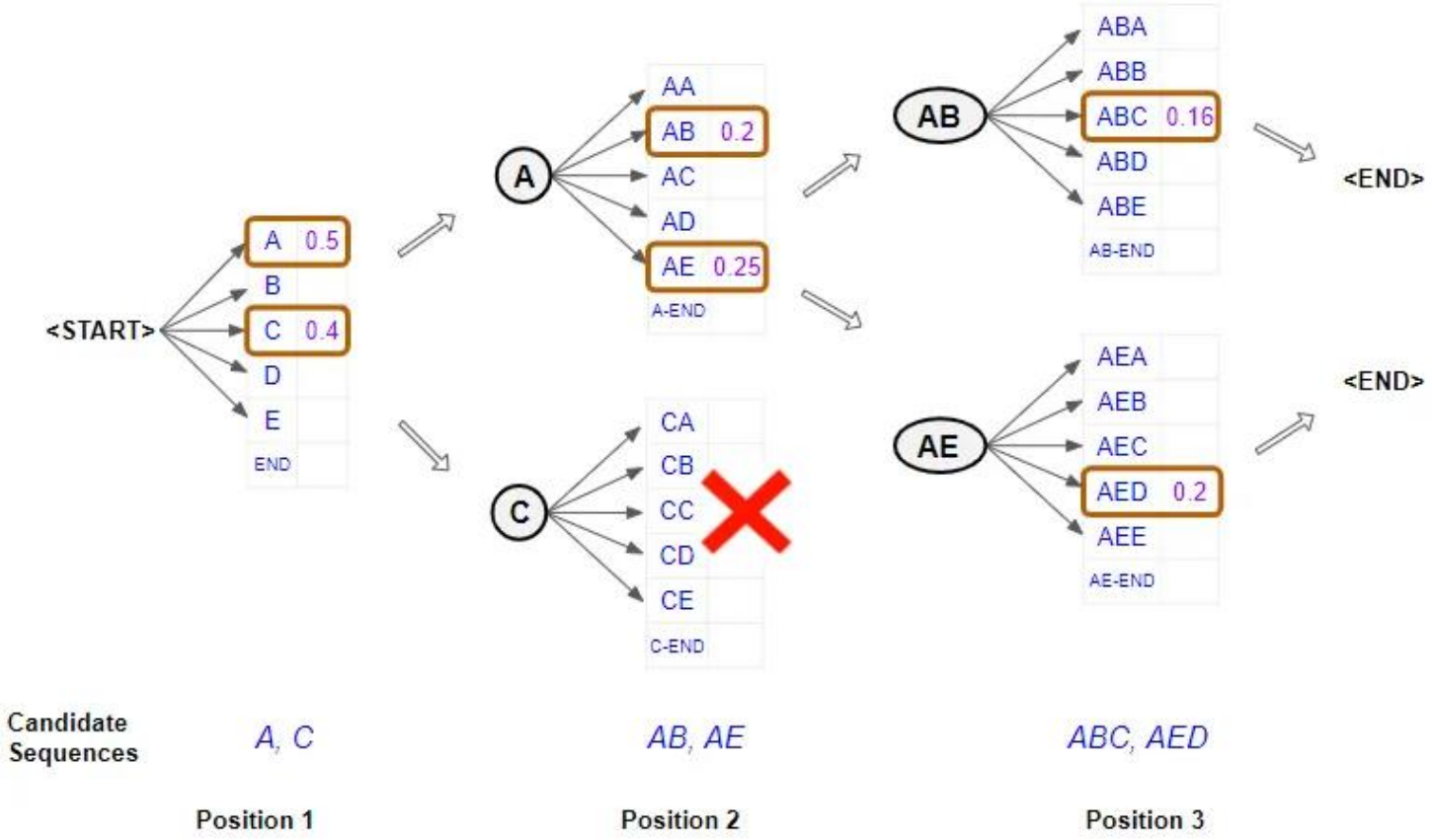
- A popular Algorithm to determine the final result of an NLP model output
- The output of NLP models is usually probilities
- At each position in sequence, which one should we choice?

Vocab				
A	0.12	0.09	...	0.82
B	0.05	...	...	0.07
...	...	...		...
Z	0.03	...	...	...
END	0.01	...	...	...
Position:	1	2	...	n

1. Problem - Background

Beam Search

- **Keep N best sequences at each step**



Beam Search with N=2

1. Problem

Background:

- Beam Search
- **Batching Techniques**
- Transformer Structure (Self-Attention)
- LLL Service & Autoregressive Generation (KV Cache)

1. Problem

Batching Techniques

- GPU specifically designed for parallel processing
- In training model, input a “batch” of number samples instead of 1 sample, the number samples is batch size
- But in inference
  - the request come one by one real-time, the server will need wait enough request for a batch then run inference
  - Each samples in a batch has different time to done but need to wait all done to return result
  - → Increase latency

(a distributed serving system called ORCA proposed a scheduler to solve this problem)

1. Problem

Background:

- Beam Search
- Batching Techniques
- **Transformer Structure (Self-Attention)**
- LLL Service & Autoregressive Generation (KV Cache)

1. Problem – Background

Transformer Structure (Self-Attention)

From Paper: *Attention Is All You Need* (Google 2017)

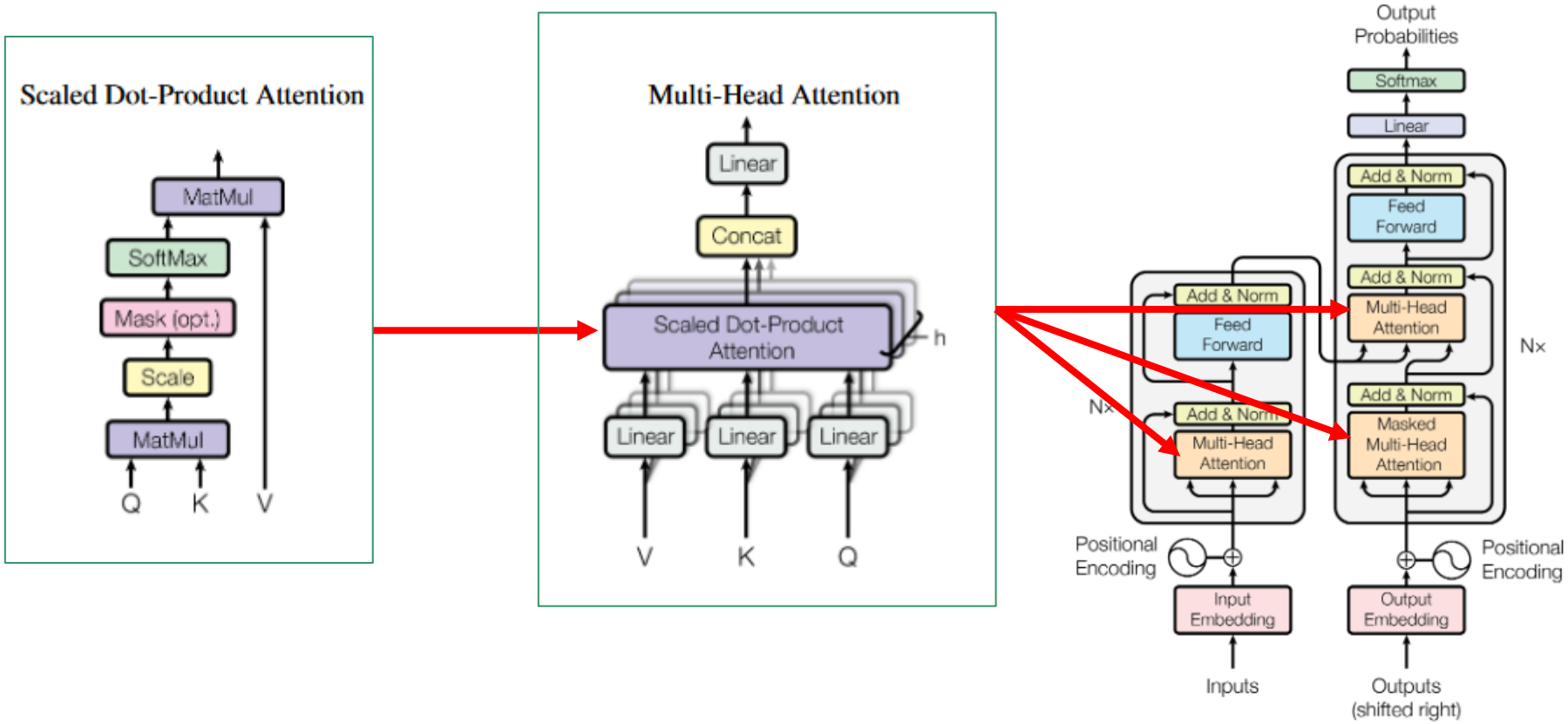
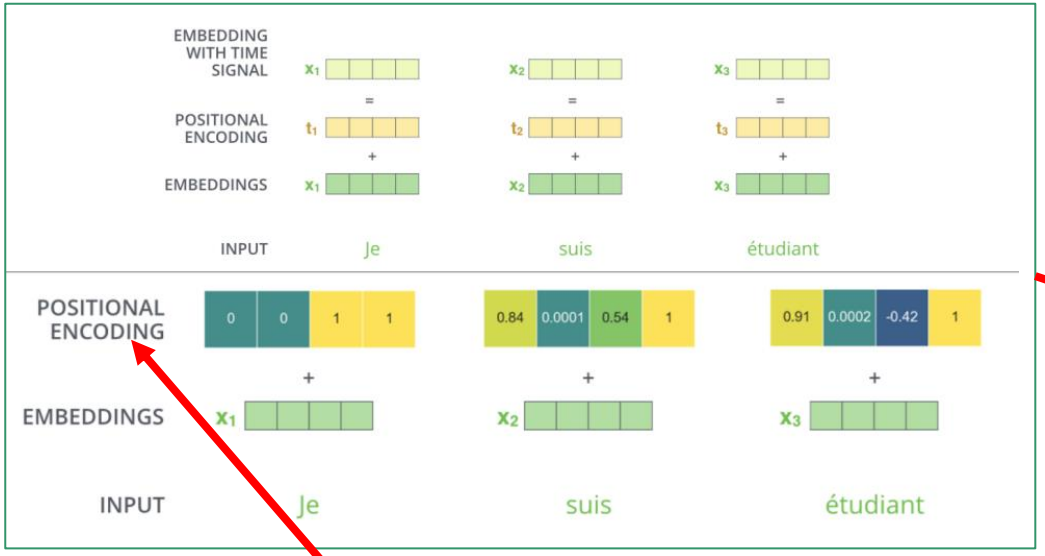


Figure 1: The Transformer - model architecture.

1. Problem – Background

From Input (word) to Embedding:

- One-hot encodings
- TF-IDF
- Word2vec (CBOW, Skip-gram)
- GloVe
- BERT



$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{model}})$$

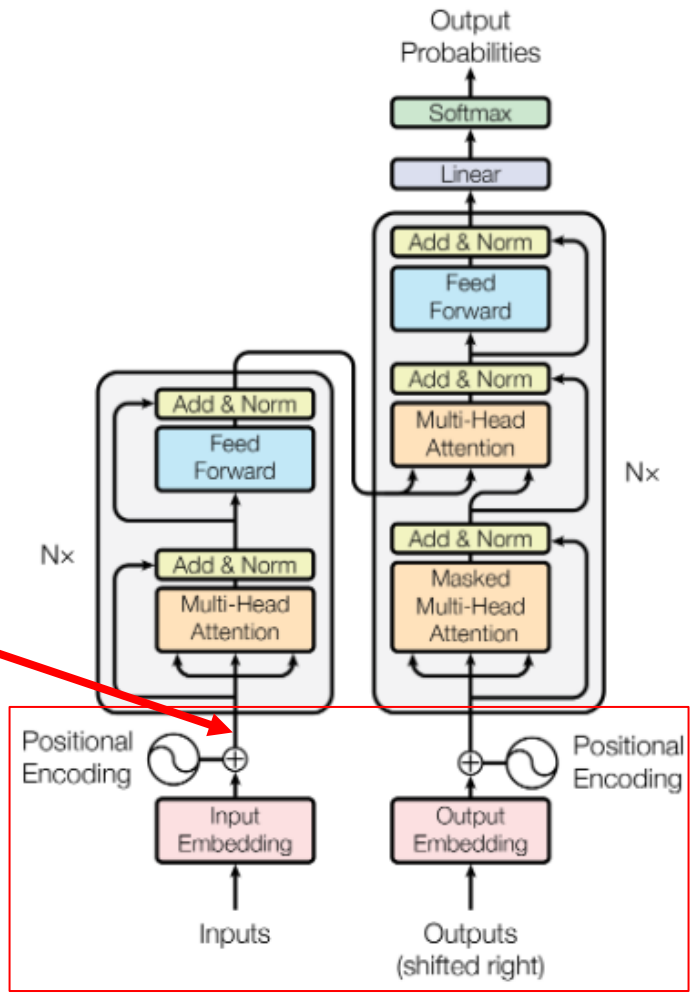


Figure 1: The Transformer - model architecture.

1. Problem – Background

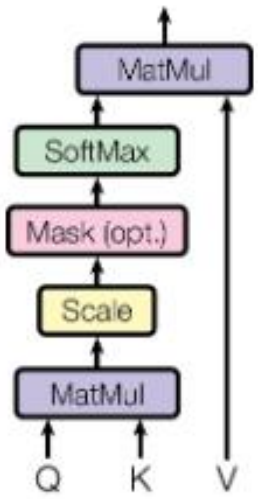
Self Attention: $$\boldsymbol{Attention(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V}$$

$\boldsymbol{d_{model}}$**: length of embeddings**

$\boldsymbol{d_k}$**: dimension of Q and K**

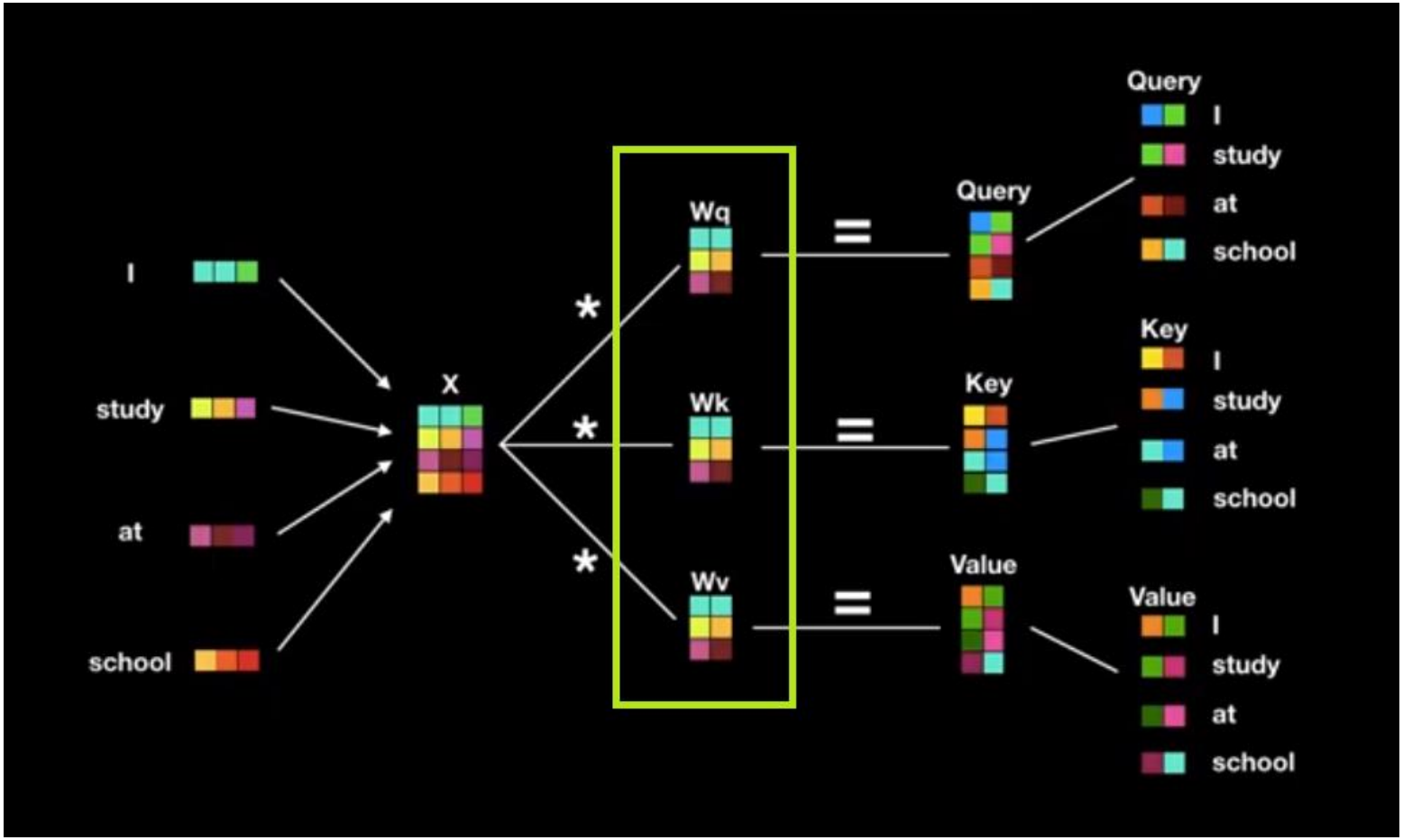
$\boldsymbol{d_v}$**: dimension of V**

Scaled Dot-Product Attention



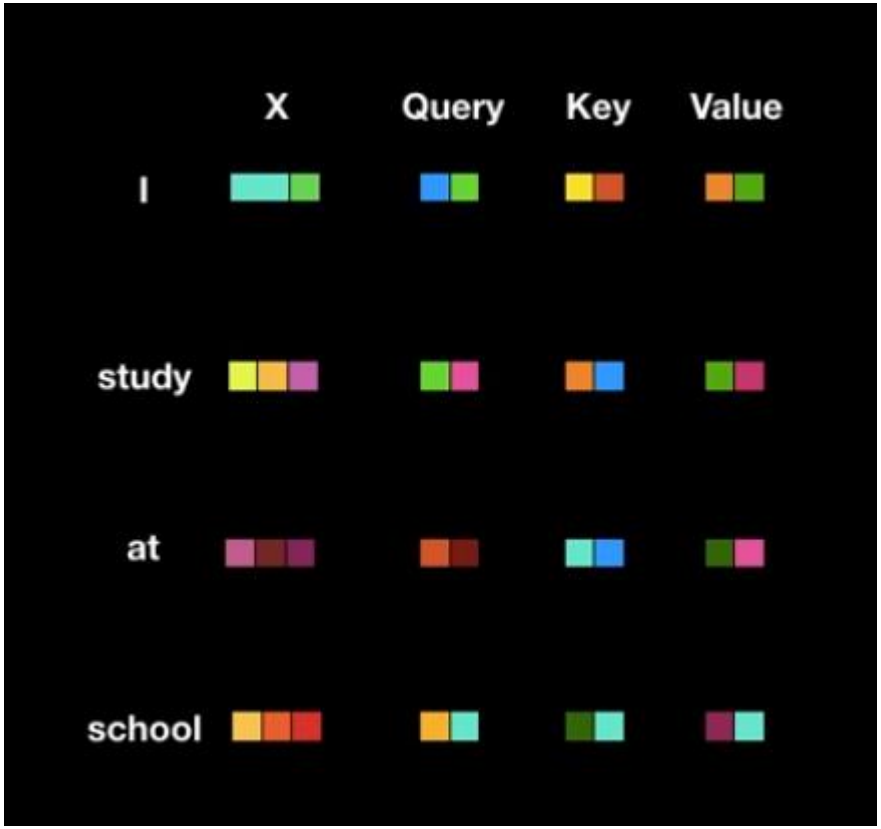
1. Problem – Background

W_Q, W_K, W_V are parameters matrix (learned by training)



1. Problem – Background

Query, Key and Value of each embeddings are calculated

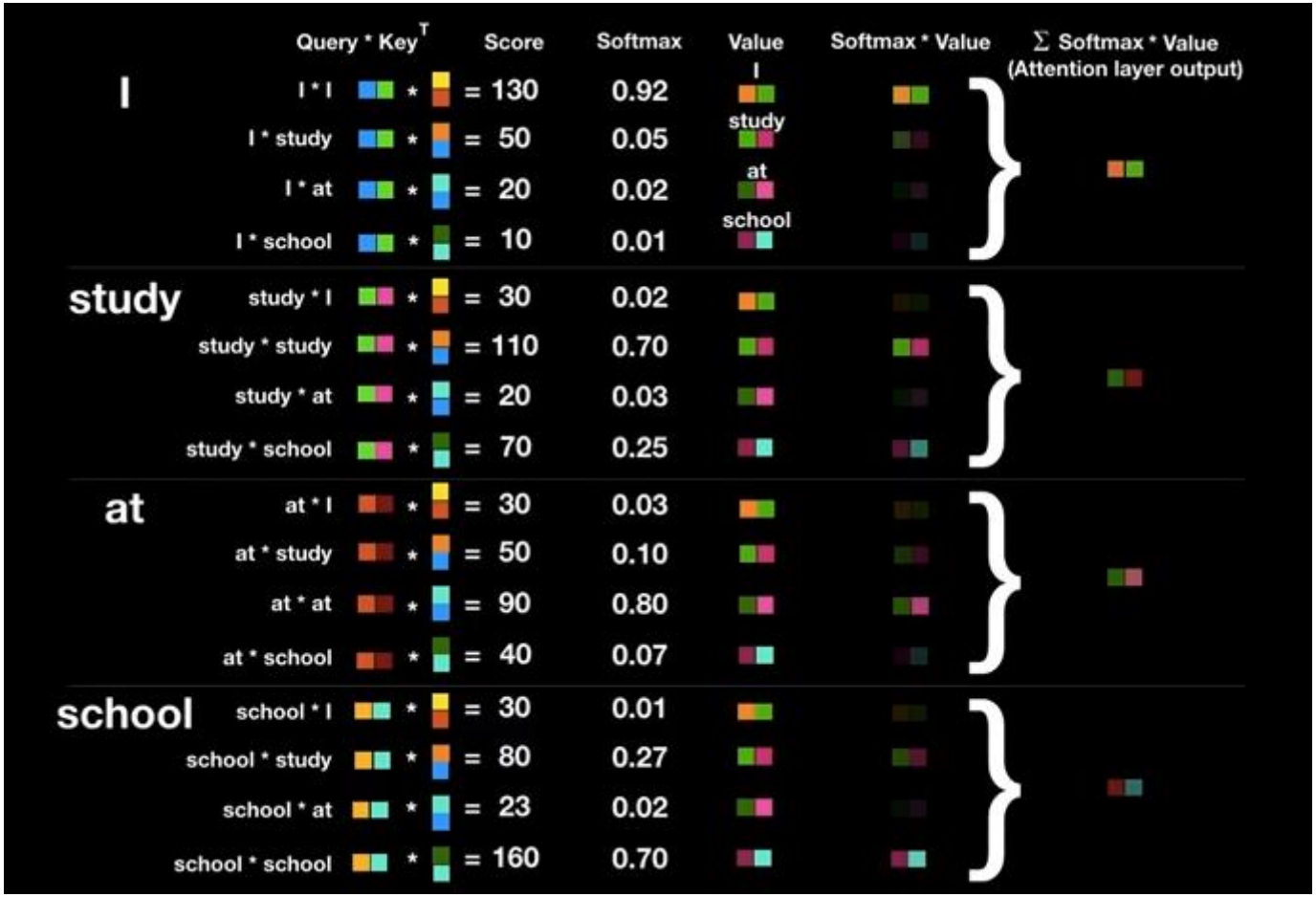


1. Problem – Background

Attention Value of Each Embeddings is calculated

1. Problem – Background

Attention Value of Each Embeddings is calculated



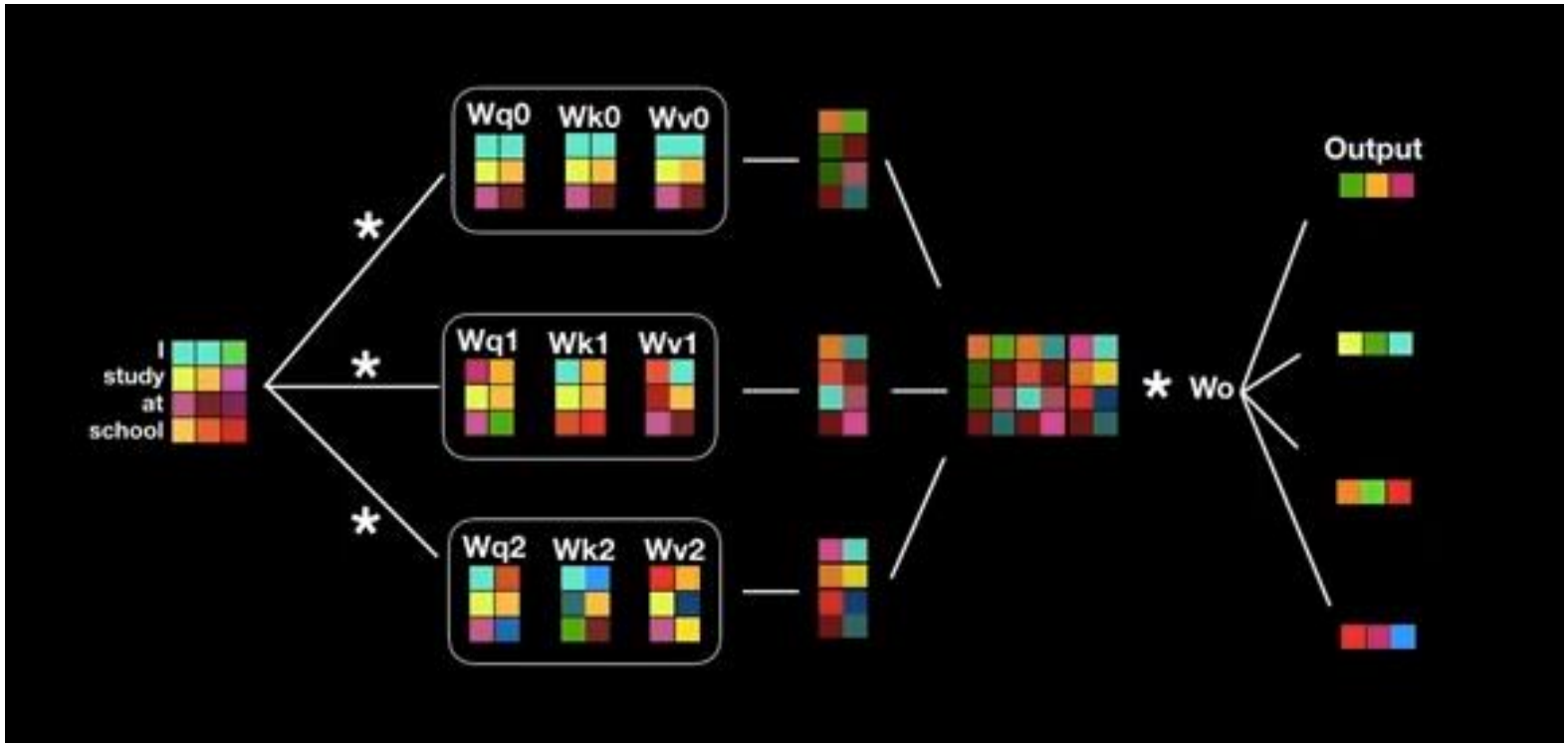
1. Problem – Background

Multi-head Attention (MHA)

$$MultiHead(Q, K, V) = concatenate(head_1, head_2, \dots head_h)W^O$$

With $head_i = Attention(Q_i, K_i, V_i)$

Matrix W^O **is used to keep output has same dimensions as input**



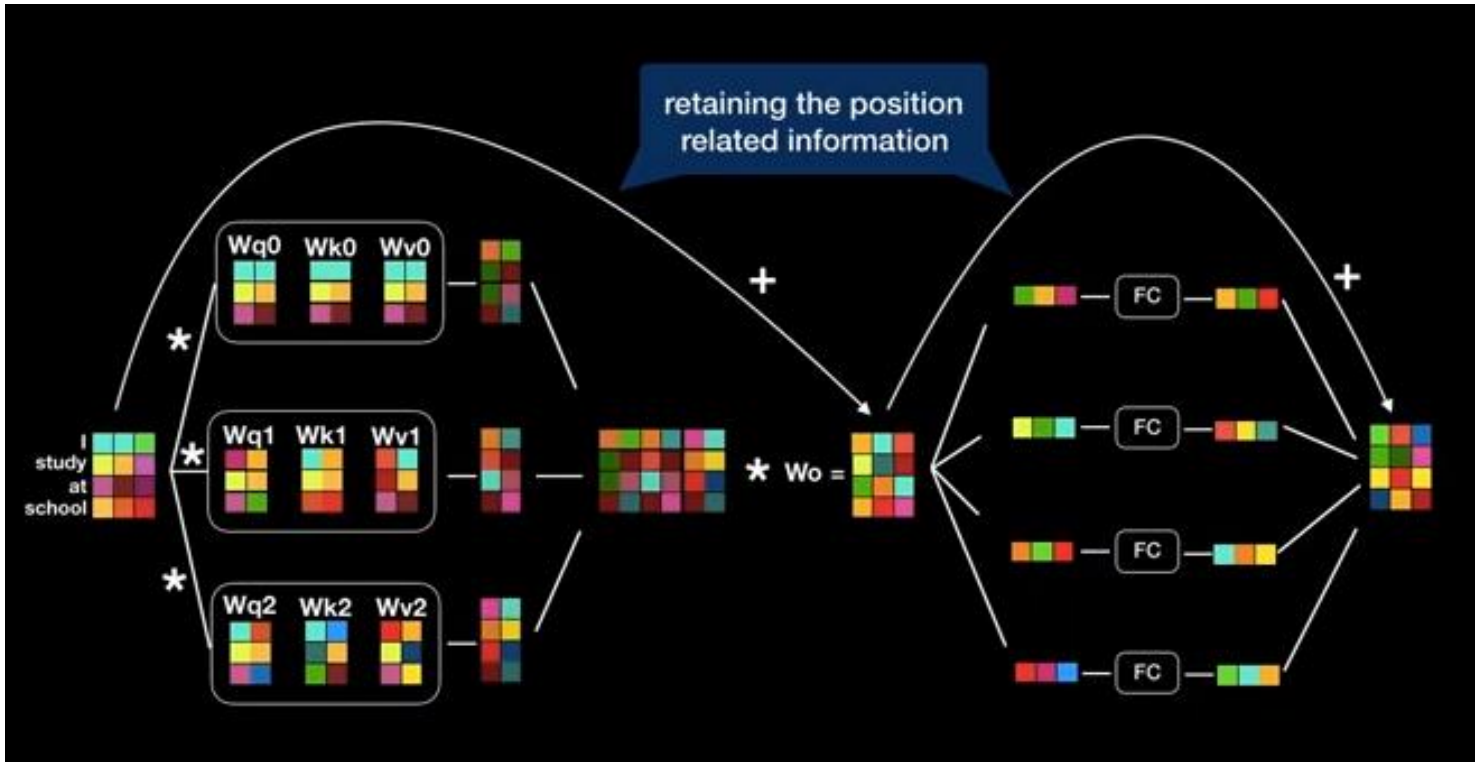
1. Problem – Background

A Block Layer:

Multi-head + Residual connection

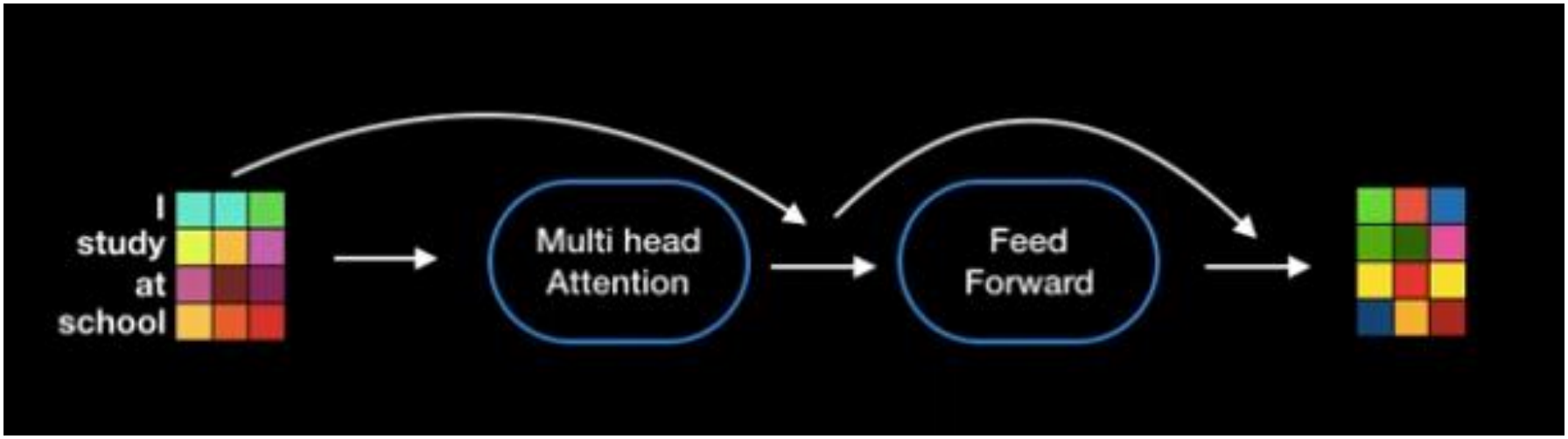
→ Normalization

→ Fully-Connected + Residual connection



1. Problem – Background

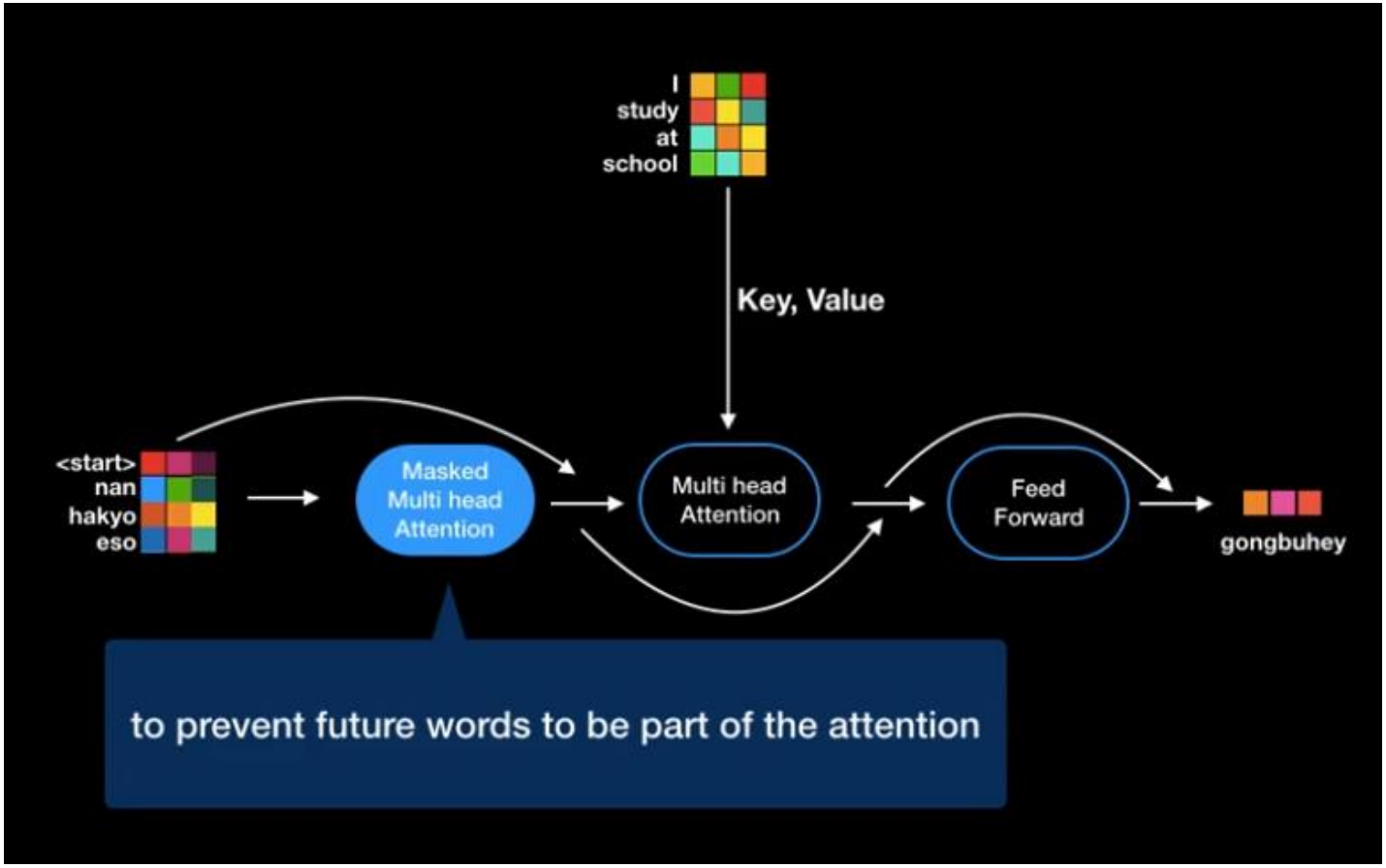
Encoder Layer



1. Problem – Background

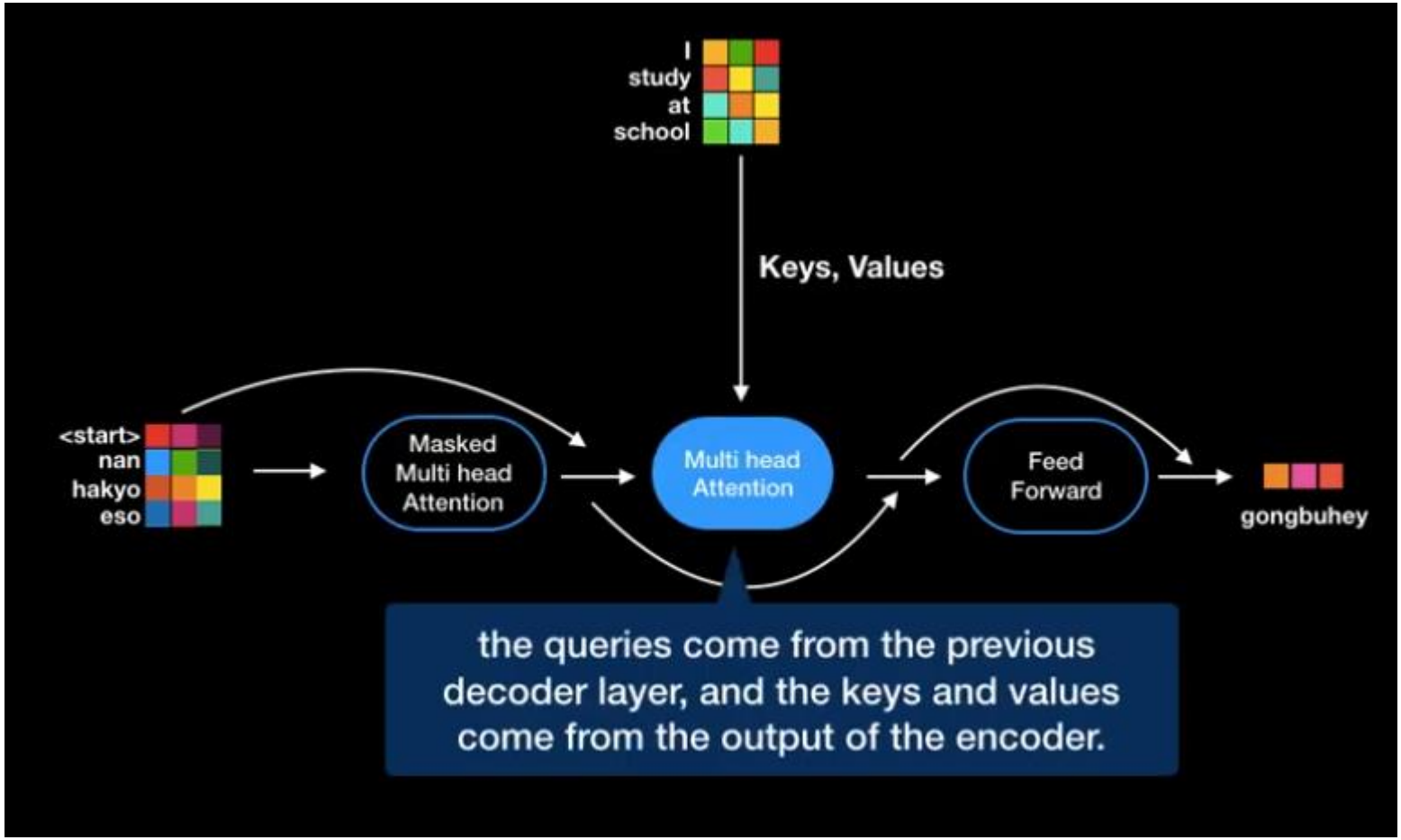
Decoder Layer

(Masked MHA don't calculate for Attention for future words)



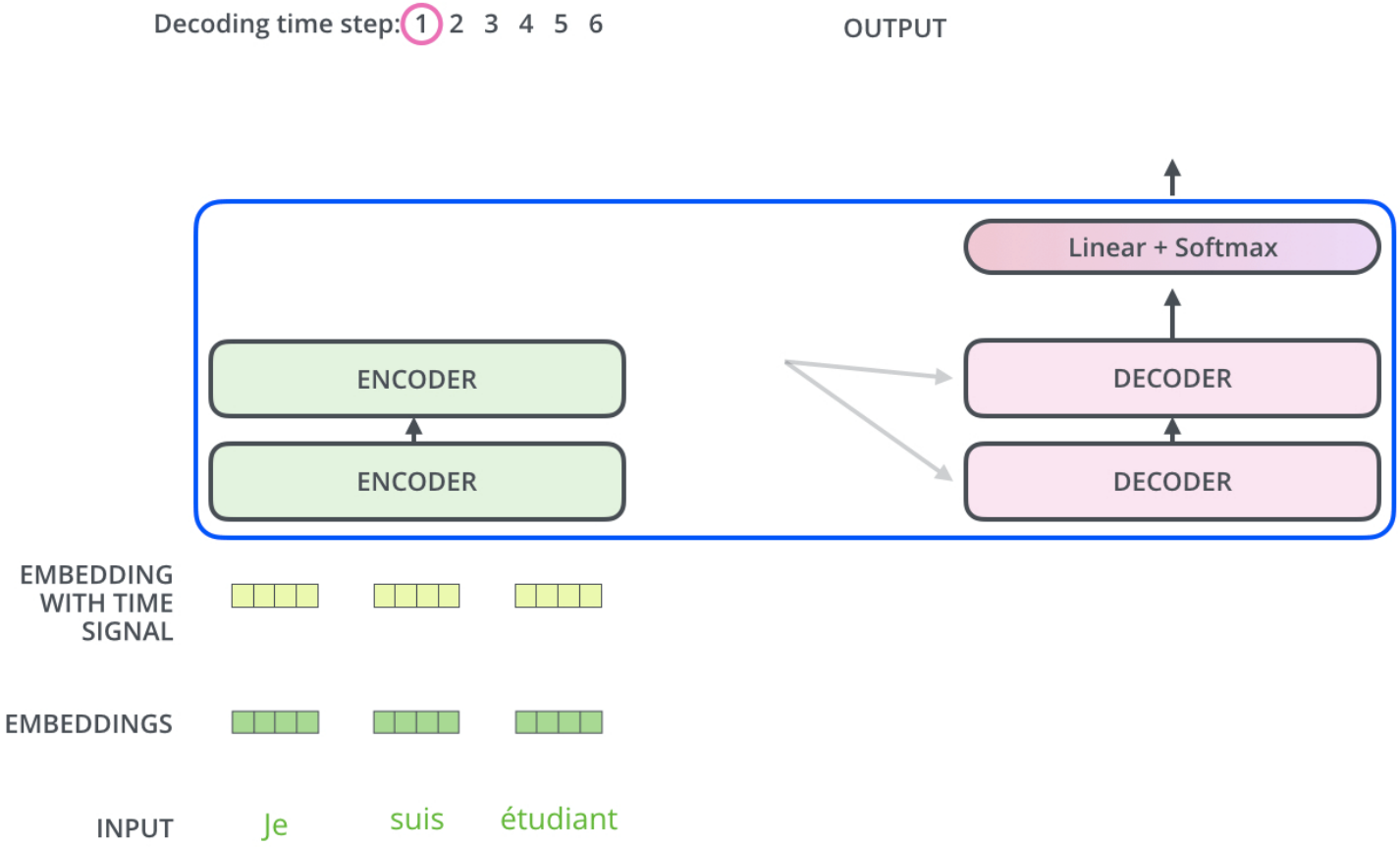
1. Problem – Background

Decoder Layer



1. Problem – Background

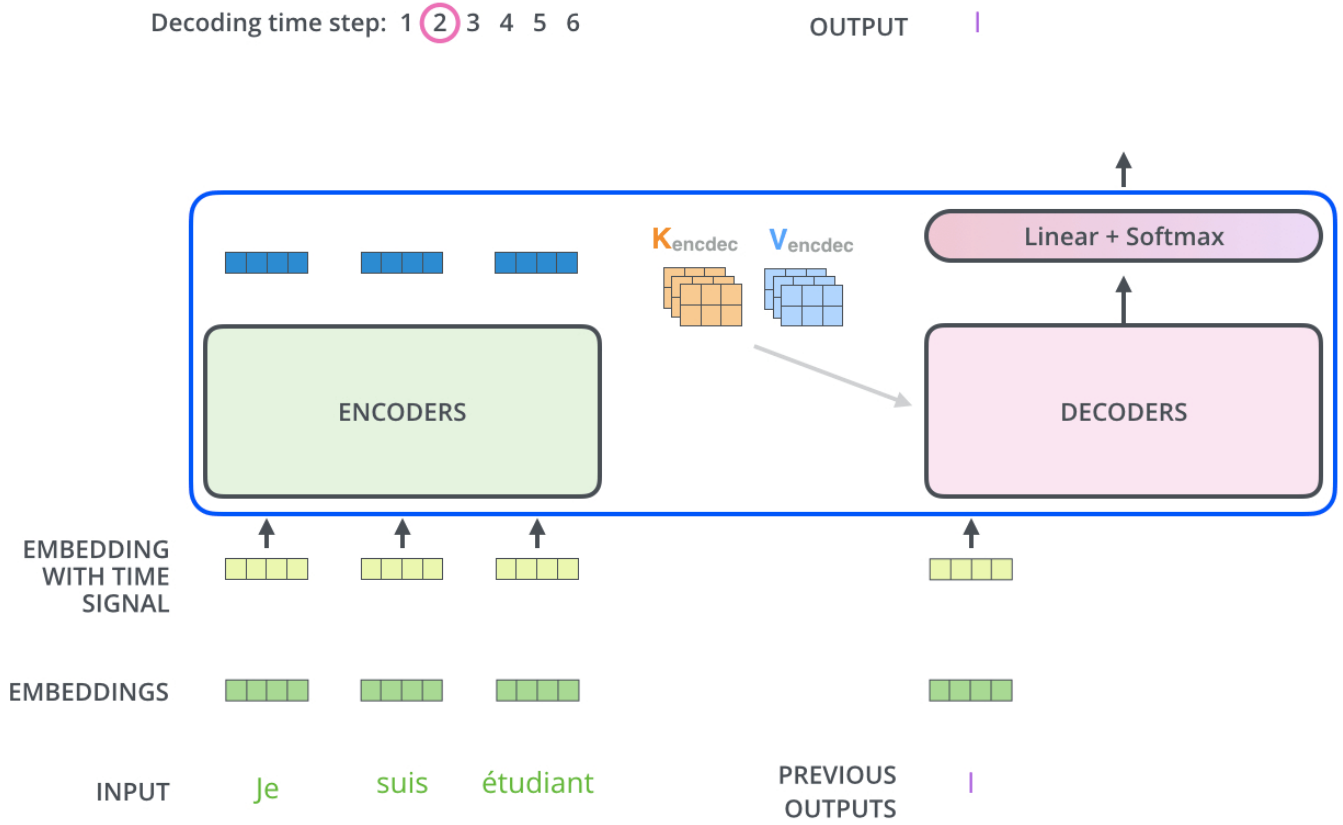
Decoder Layer



1. Problem – Background

Decoder Layer, the generation will end in 2 cases:

- **The max sequences length**
- **Meet the <eos> output**



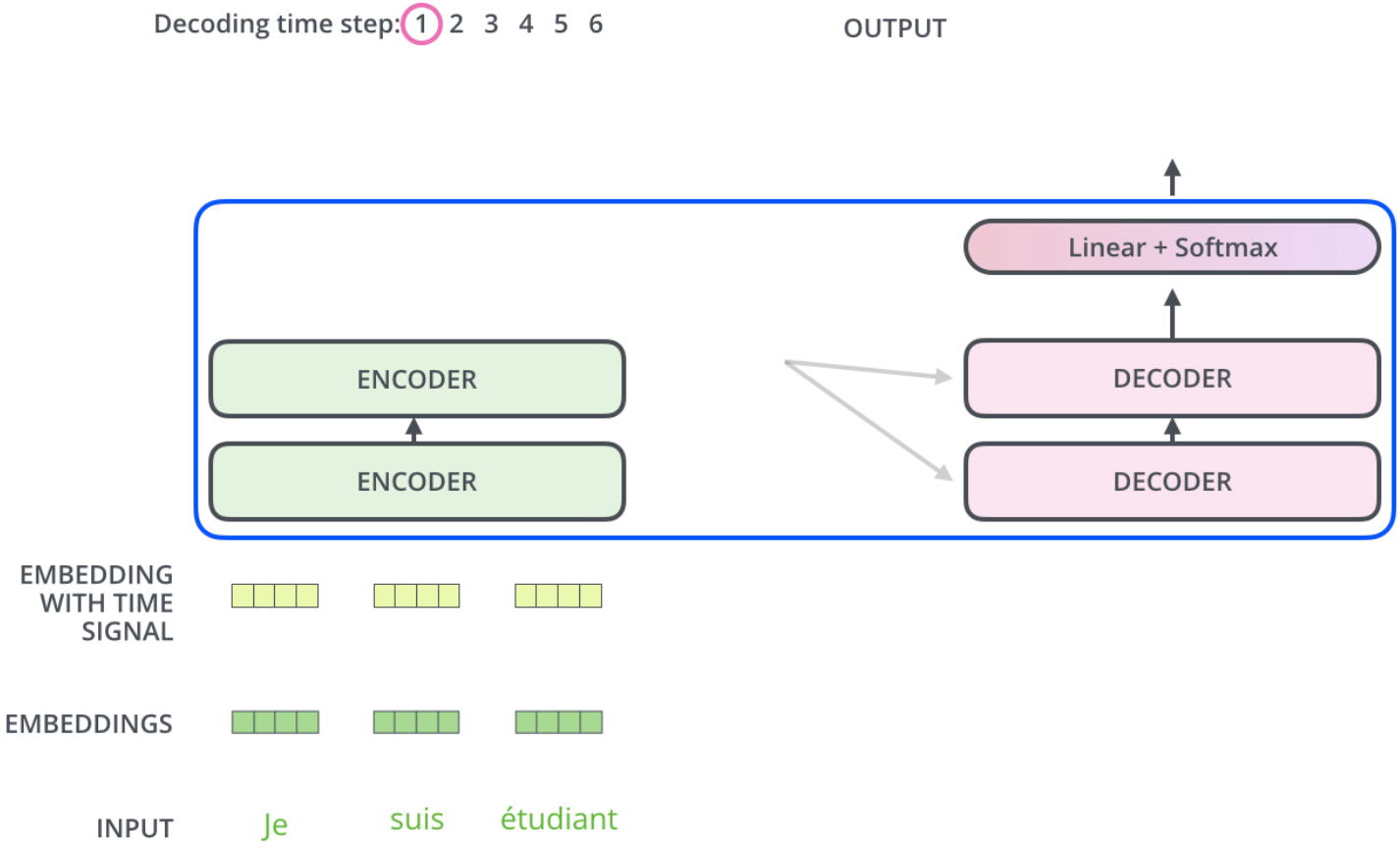
1. Problem

Background:

- Beam Search
- Batching Techniques
- Transformer Structure (Self-Attention)
- **LLL Service & Autoregressive Generation (KV Cache)**

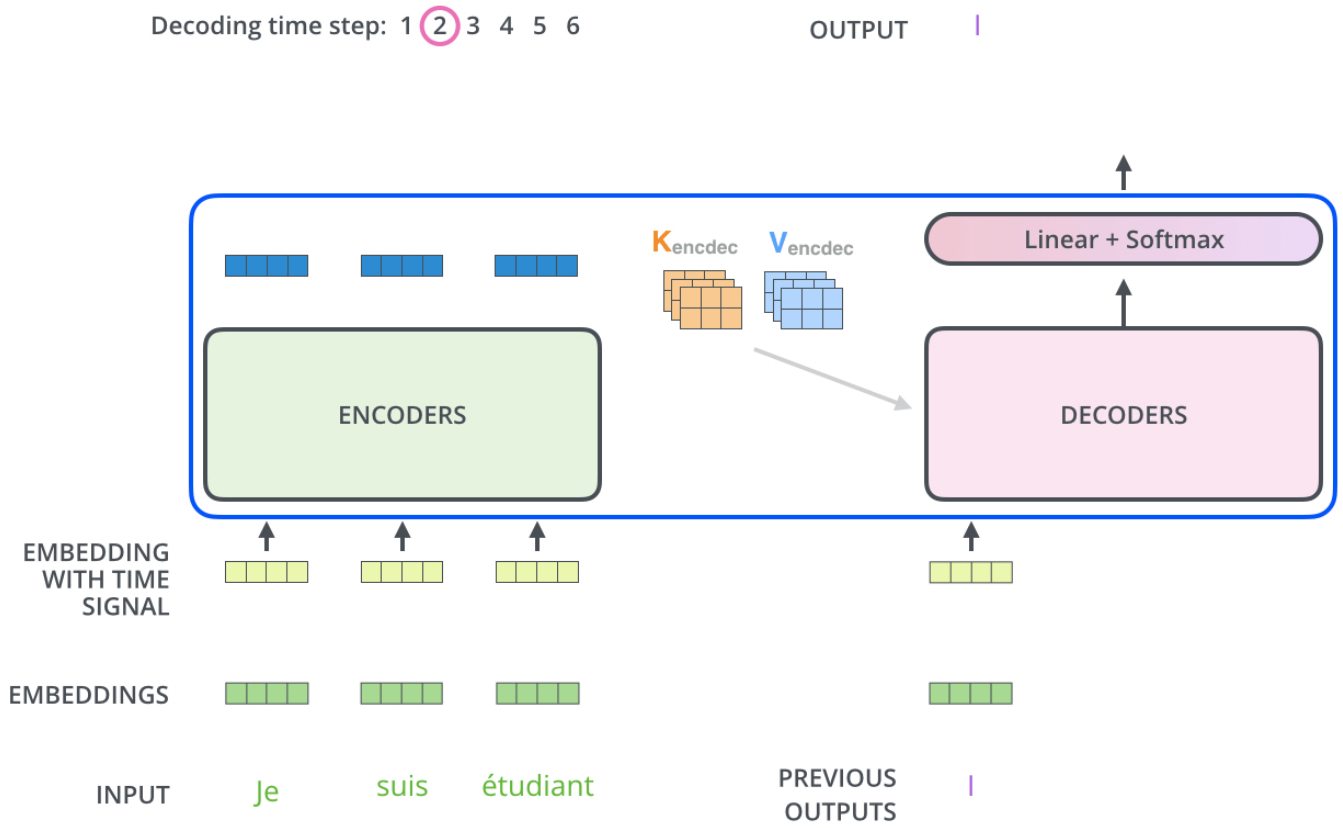
1. Problem – Background

Inference Initialize



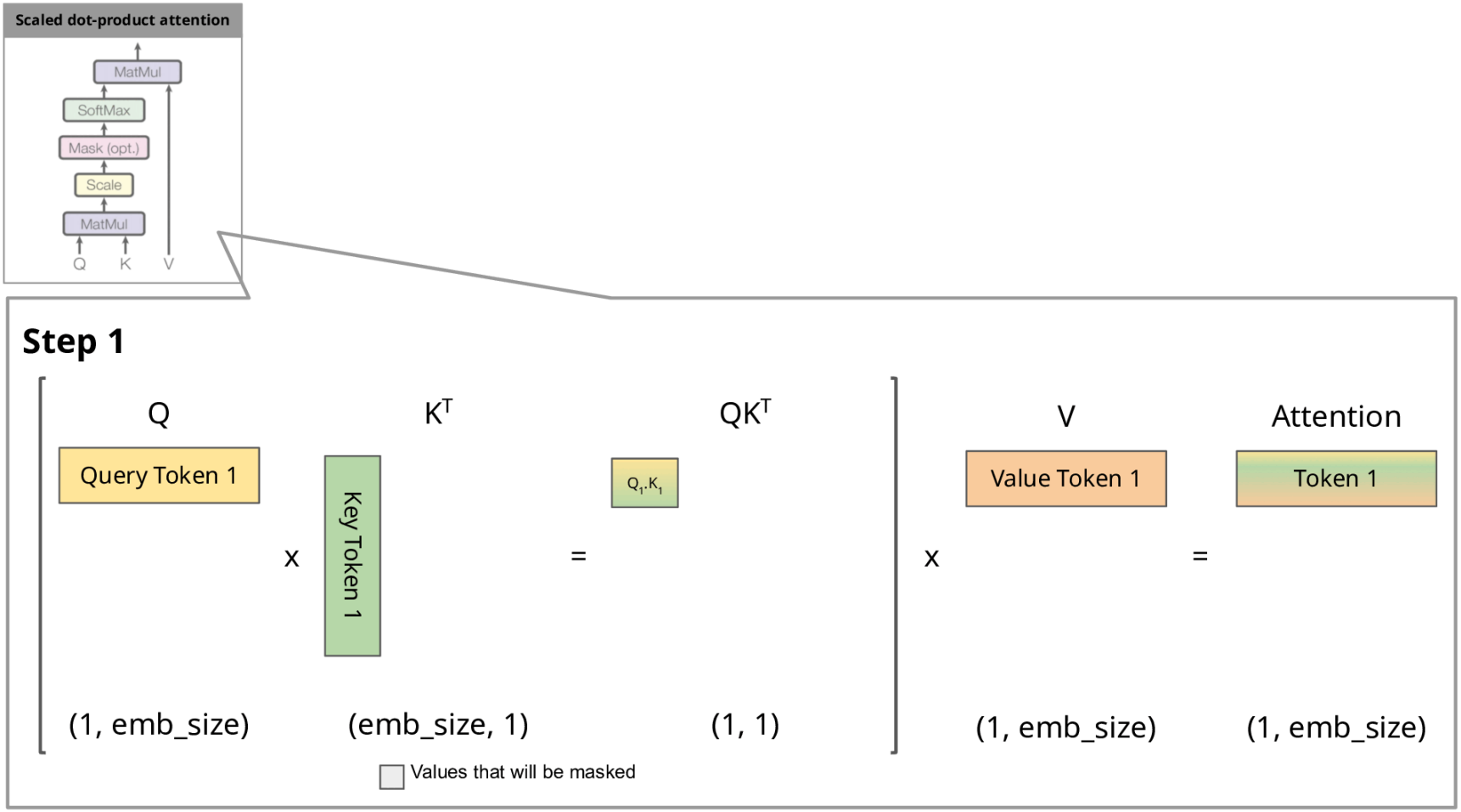
1. Problem – Background

Autoregressive Generation



1. Problem – Background

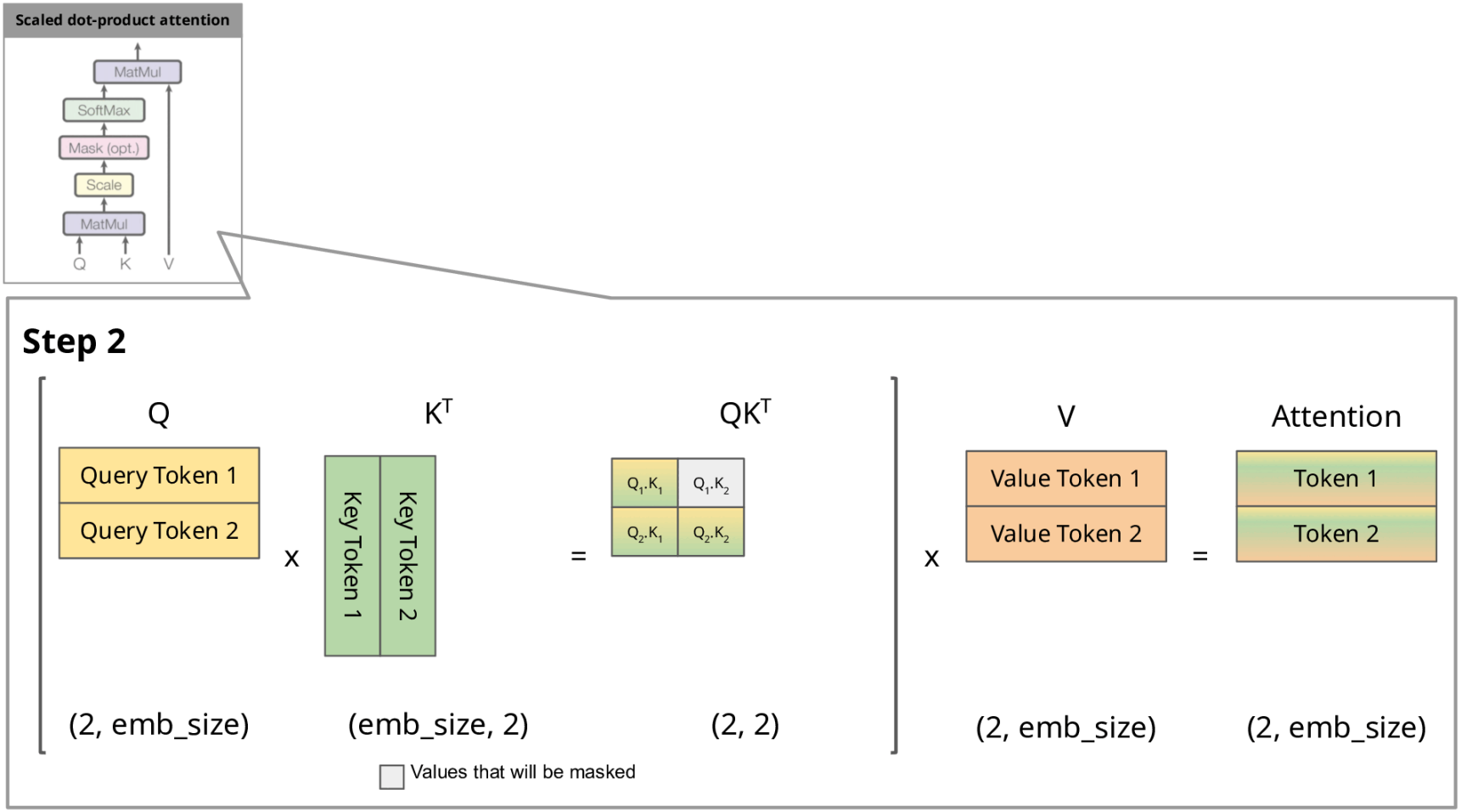
Auto-regressive Calculations in masked scaled dot-product attention



Zoom-in! (simplified without Scale and Softmax)

1. Problem – Background

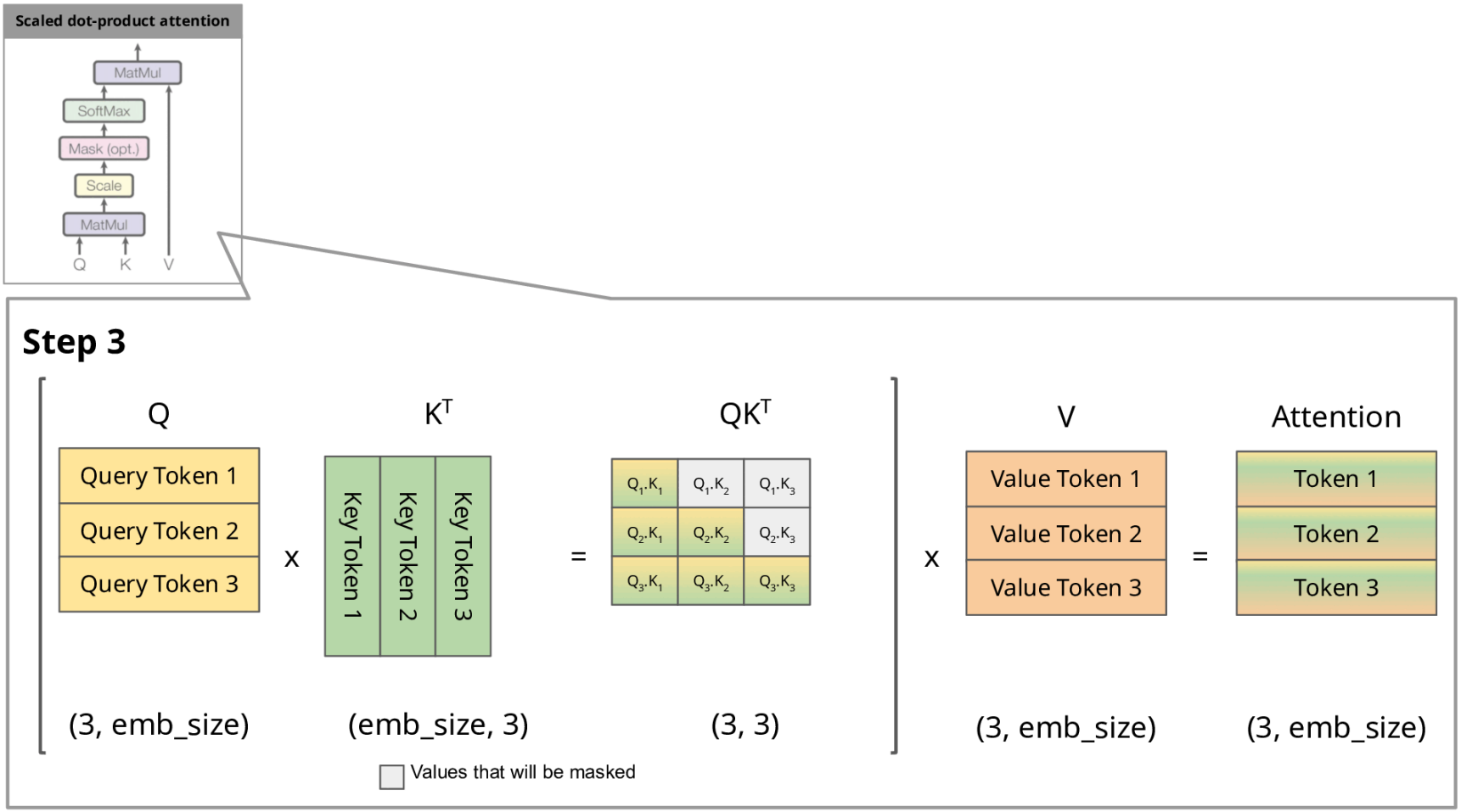
Auto-regressive Calculations in masked scaled dot-product attention



Zoom-in! (simplified without Scale and Softmax)

1. Problem – Background

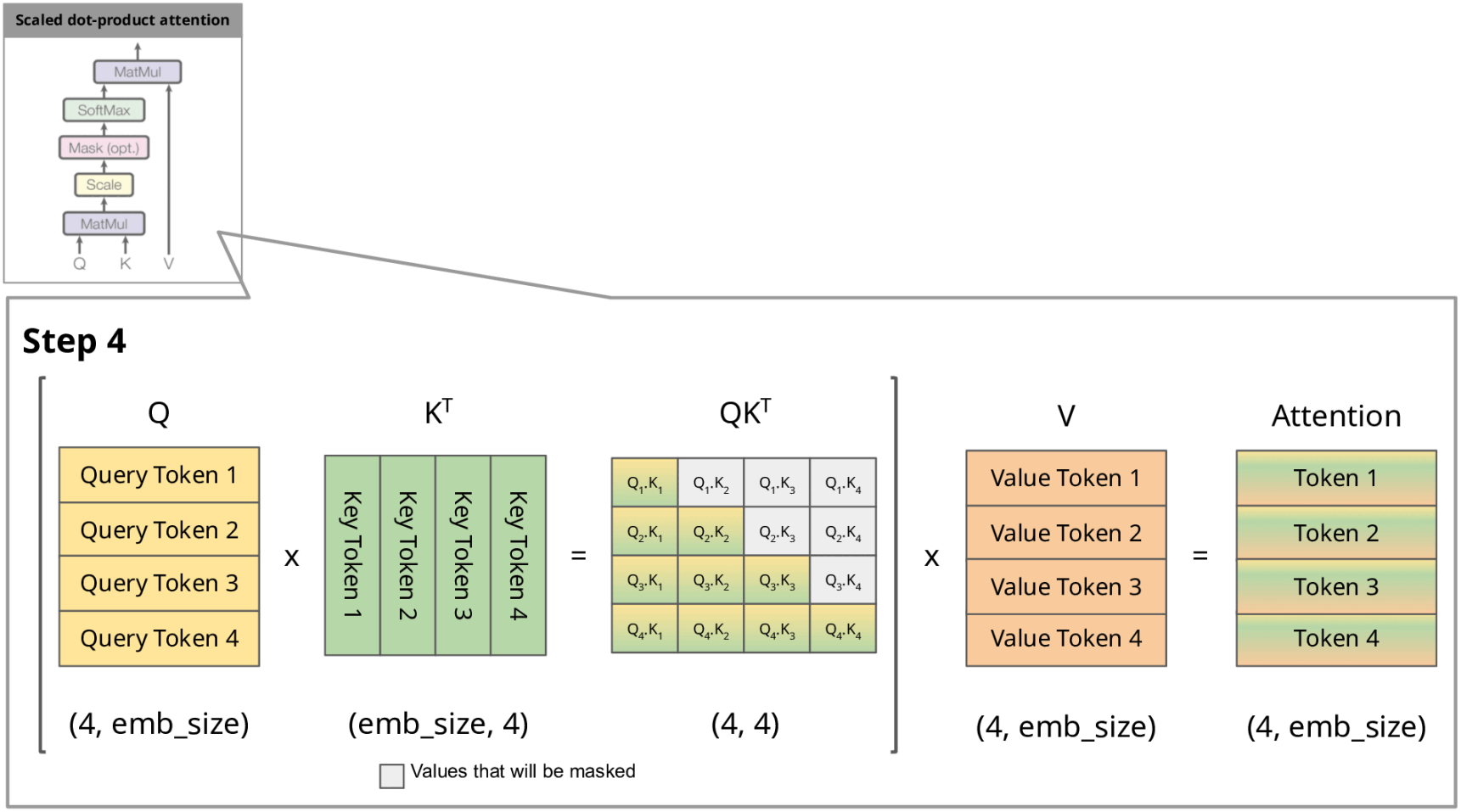
Auto-regressive Calculations in masked scaled dot-product attention



Zoom-in! (simplified without Scale and Softmax)

1. Problem – Background

Auto-regressive Calculations in masked scaled dot-product attention



Zoom-in! (simplified without Scale and Softmax)

1. Problem – Background

FLOPs required to compute the attention scores:

$$\boldsymbol{O(2 * b * n_{layers} * d_{model} * t^2)}$$

with b is batch_size, t is total sequence length

→ **quadratic scaling of attention computation by sequence length**

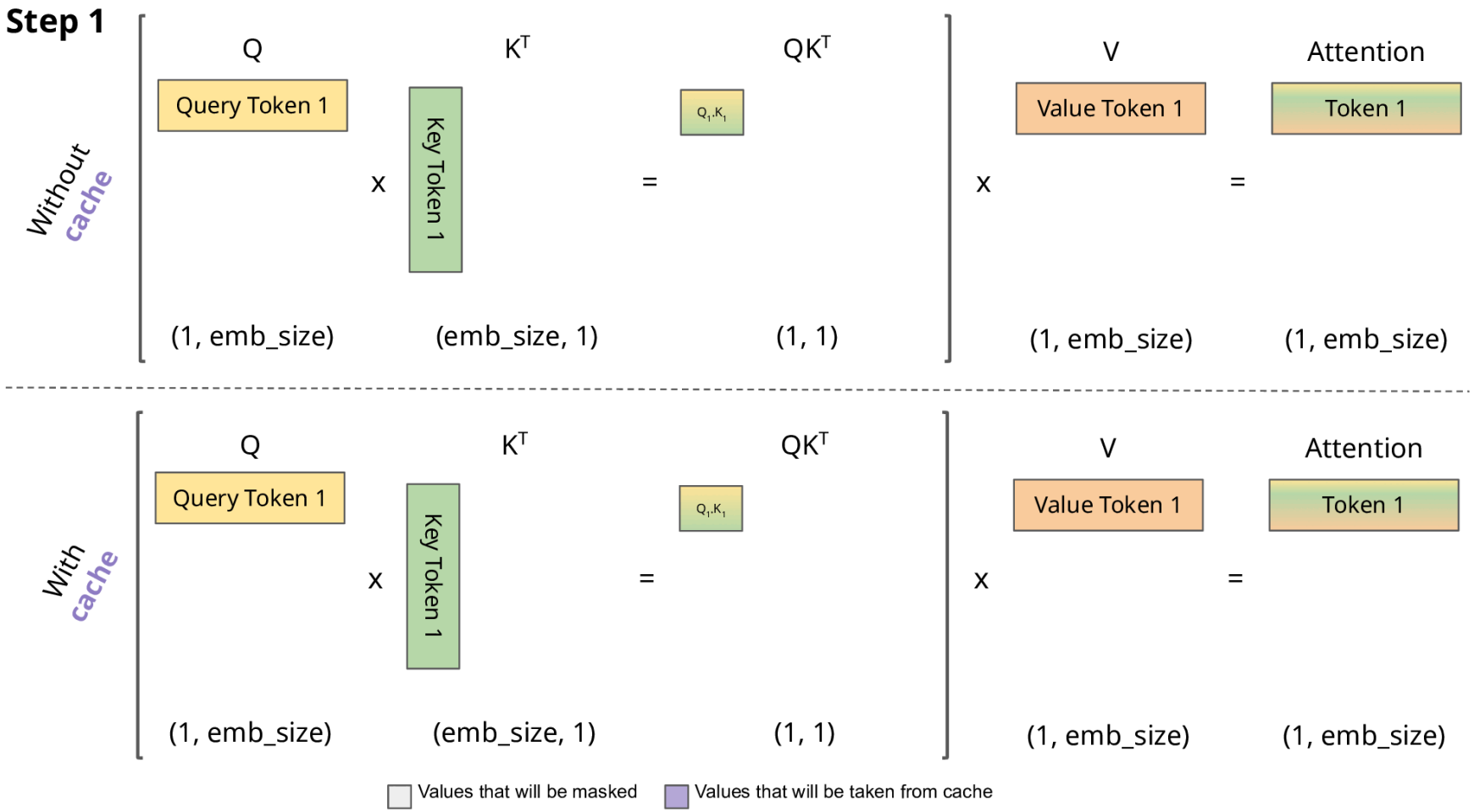
To generate the 1,001st token, the model must perform 100x more FLOPs than to generate the 101st token

→ **Using KV Cache, a trade-off between Memory Usage and Computation**

- Cache the Keys and Values on GPU/CPU VRAM for the next calculating the attention for the new token

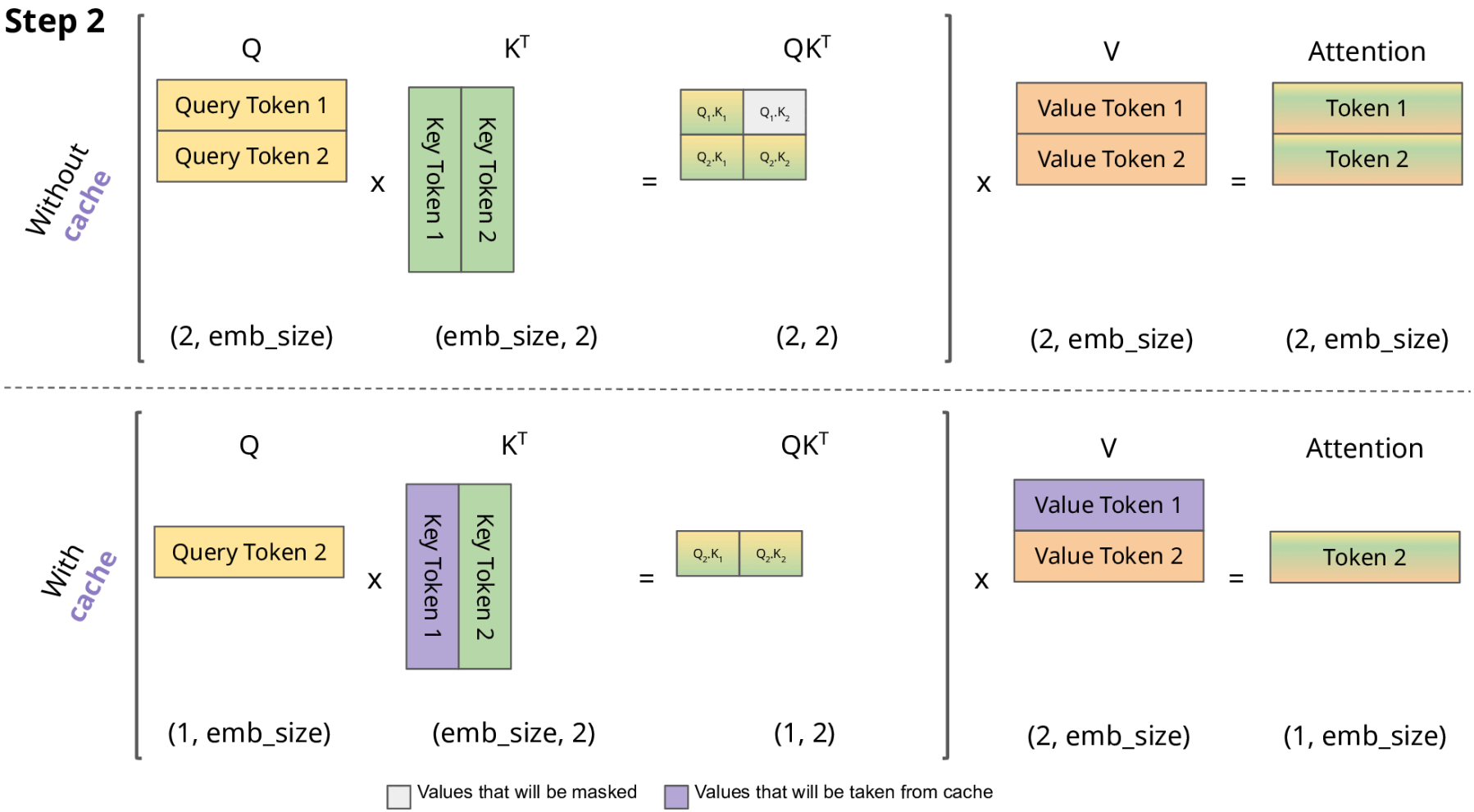
1. Problem – Background

KV Cache



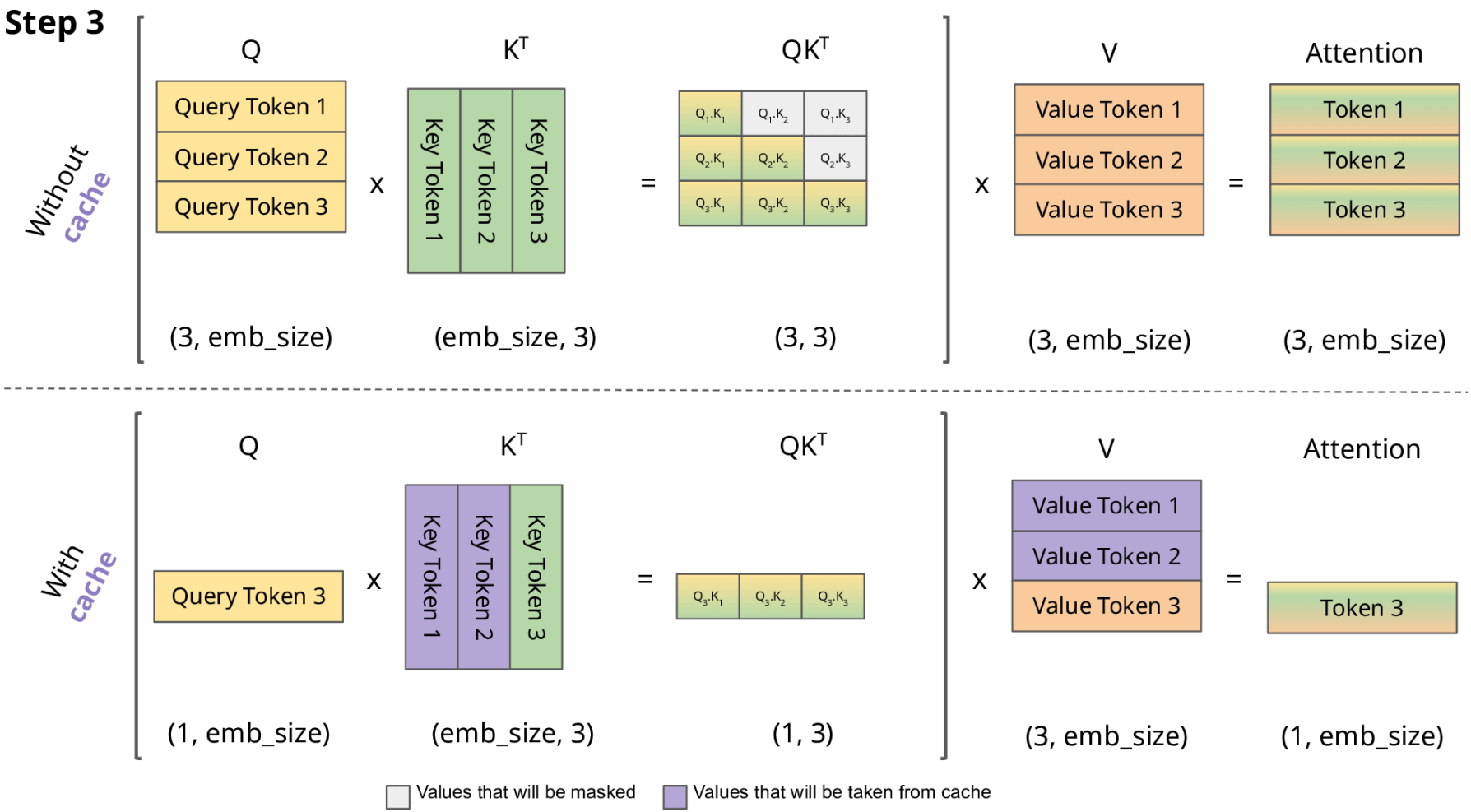
1. Problem – Background

KV Cache



1. Problem – Background

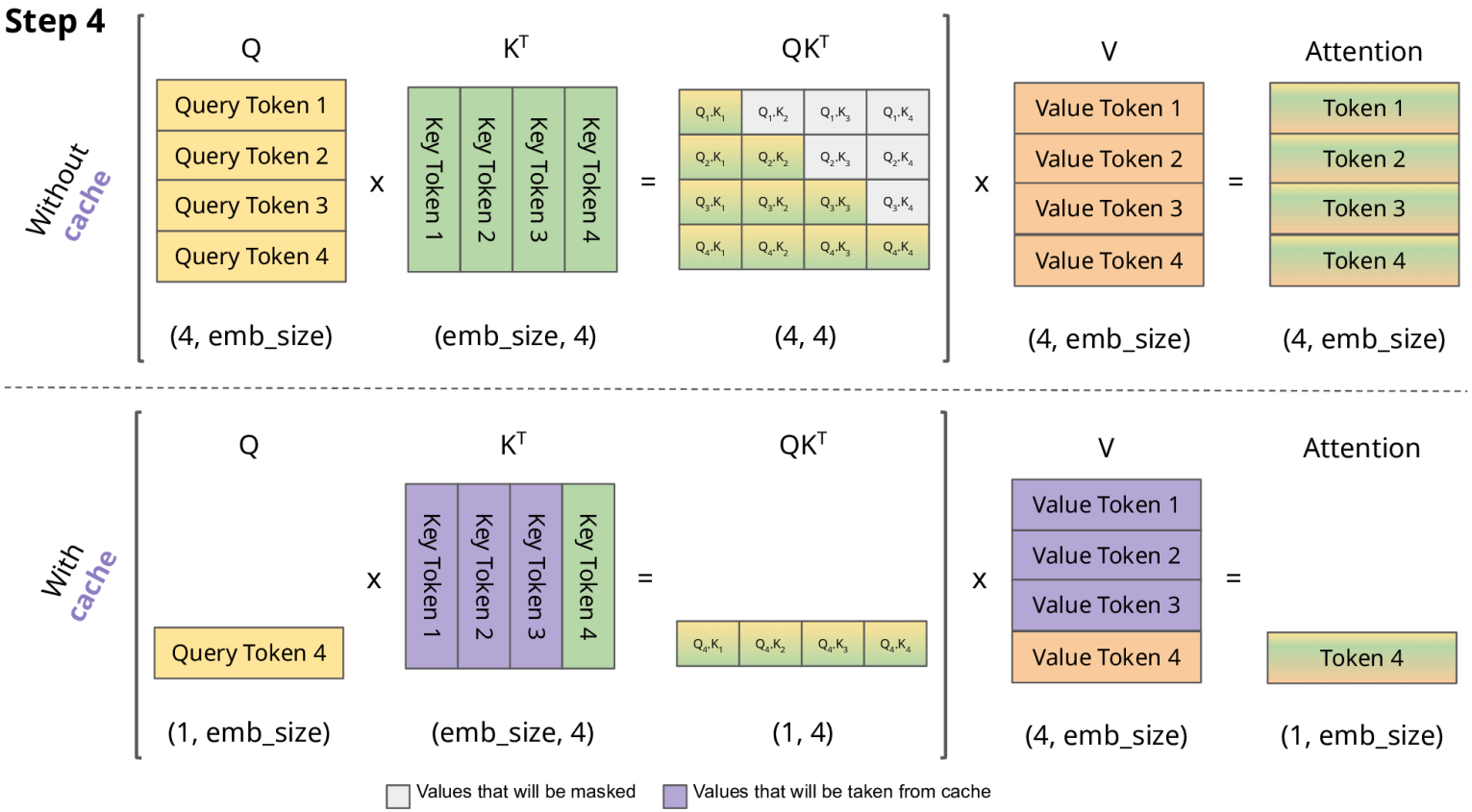
KV Cache



1. Problem – Background

KV Cache

Estimate: Attention now scales linearly with total sequence length



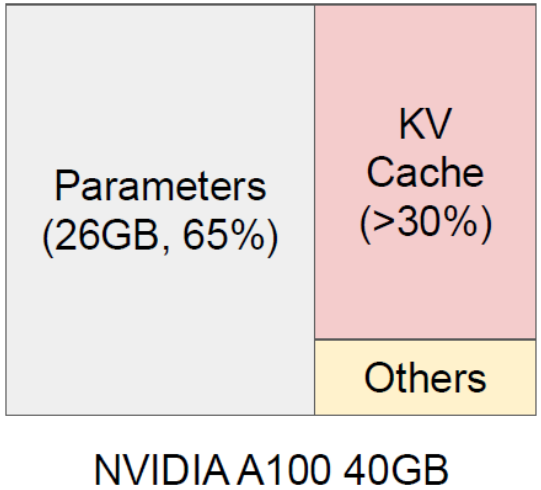
1. Problem

Determine Problems in KV Cache

1. Problem

Analyzing the GPU memory

- **65% memory for static model weights → constant**
- **About 5% for other data, including activations → small, don't matter!**
- **30% for KV Cache during serving → Can it enhance?**



Serving 13B parameters

1. Problem

Large KV Cache

- Size grows quickly with number of requests. E.g. model 13B need for a request: $2048_{tokens} * 2_{k_v_sector} * 5120_{hidden_state} * 40_{layers} *$

  = ~ 1.6 GB

  -> NVIDIA A100 can serve about 10 requests each time

- Current trend in GPU: computation speed >> memory capacity

  e.g. from NVIDIA A100 -> H100, FLOPS increase more 2x while memory stay at 80G

  → Memory bottleneck

	A100 80GB PCIe	A100 80GB SXM
FP64	9.7 TFLOPS	
FP64 Tensor Core	19.5 TFLOPS	
FP32	19.5 TFLOPS	
Tensor Float 32 (TF32)	156 TFLOPS | 312 TFLOPS*	
BFLOAT16 Tensor Core	312 TFLOPS | 624 TFLOPS*	
FP16 Tensor Core	312 TFLOPS | 624 TFLOPS*	
INT8 Tensor Core	624 TOPS | 1248 TOPS*	

	H100 SXM	H100 PCIe
FP64	34 teraFLOPS	26 teraFLOPS
FP64 Tensor Core	67 teraFLOPS	51 teraFLOPS
FP32	67 teraFLOPS	51 teraFLOPS
TF32 Tensor Core	989 teraFLOPS[2]	756 teraFLOPS[2]
BFLOAT16 Tensor Core	1,979 teraFLOPS[2]	1,513 teraFLOPS[2]
FP16 Tensor Core	1,979 teraFLOPS[2]	1,513 teraFLOPS[2]
FP8 Tensor Core	3,958 teraFLOPS[2]	3,026 teraFLOPS[2]
INT8 Tensor Core	3,958 TOPS[2]	3,026 TOPS[2]

Serving 13B parameters on NVIDIA A100

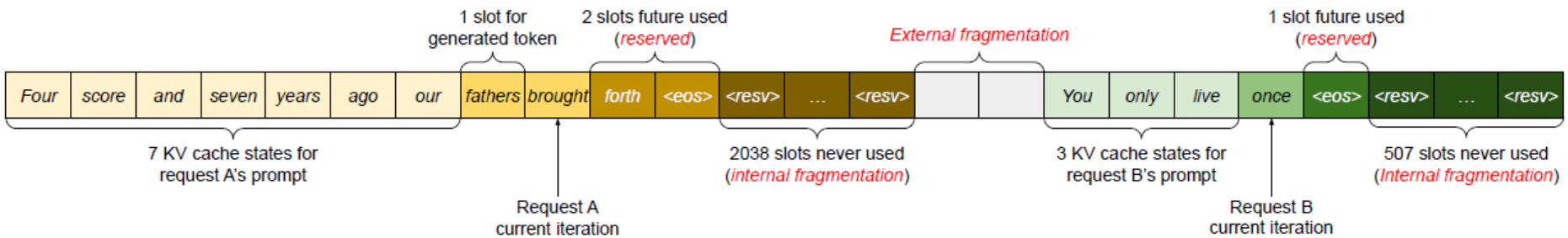
1. Problem

Before June 2023, there are two problems

- Existing LLM serving system (ORCA, FasterTransformer) also DL framework such as Pytorch, Tensorflow store K-V Cache in **continuous memory space** GPU → **pre-allocate** a continuous chunk of GPU memory with **maximum length** (e.g., 2048) – **buddy allocator**

  → **reserved, internal fragmentation and external fragmentation**

  (Only 20-40% of KV Cache is utilized)

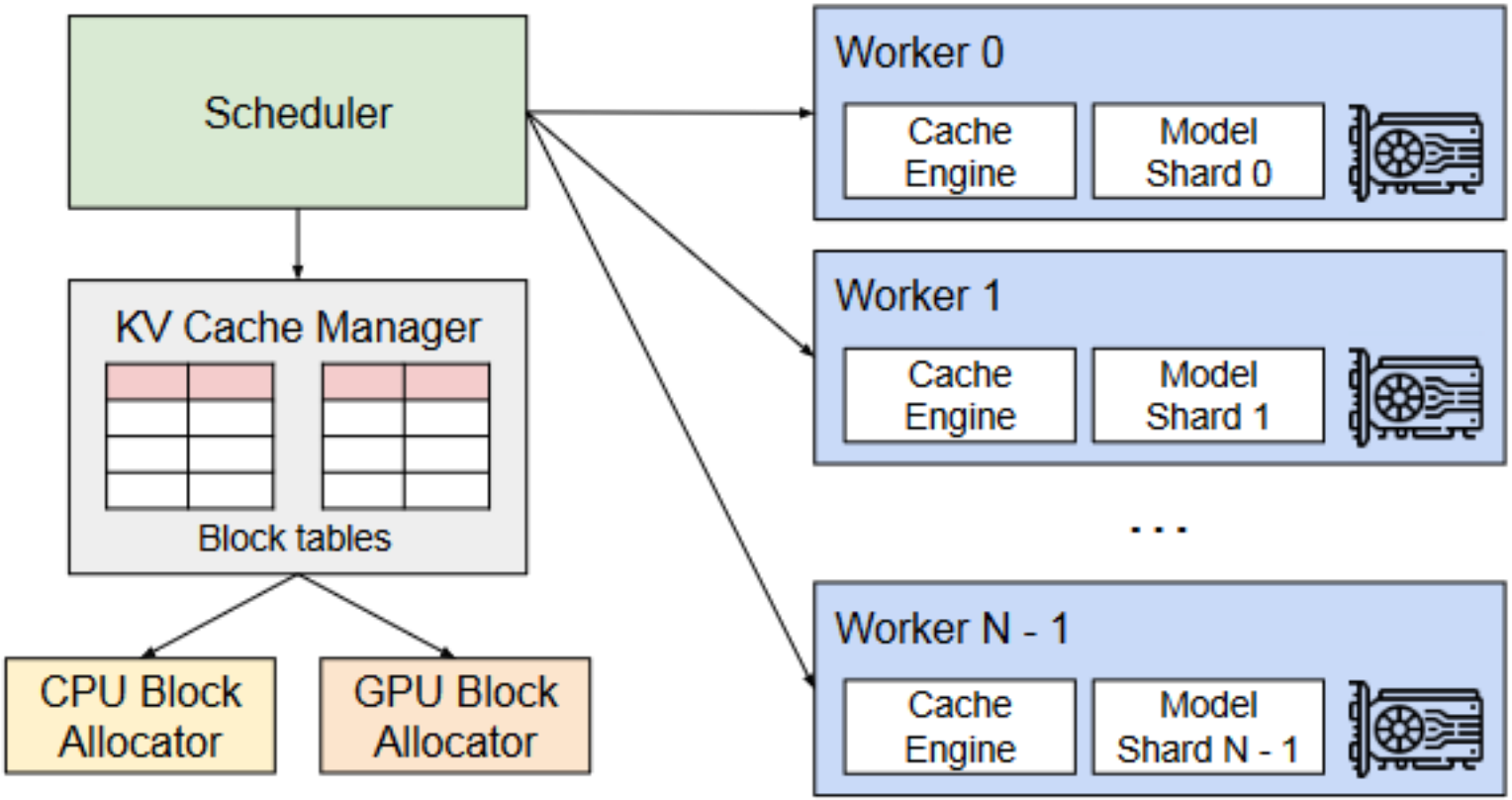


- Not sharing K-V Cache for parallel sampling (e.g., program suggestion) and beam search
- Need Scheduler for delete/swap KV Cache

PagedAttention - vLLM

1. Problems with Large Language Model (LLM) Inference System
2. **PagedAttention - vLLM**
3. Related works

PagedAttention - vLLM



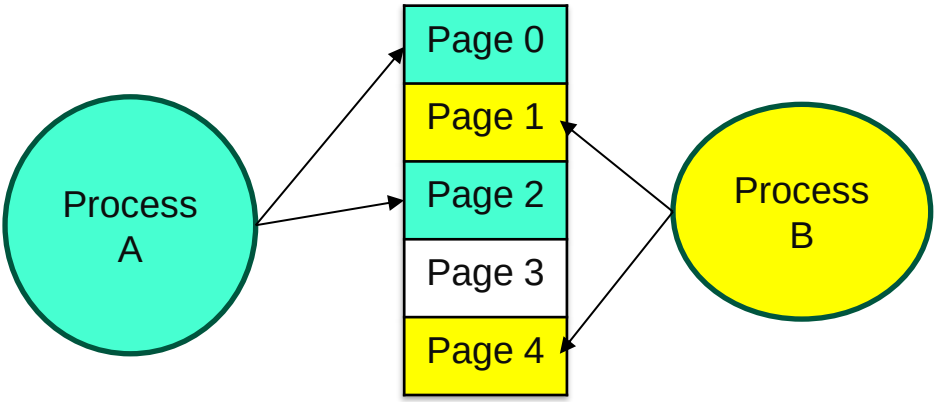
vLLM system overview

PagedAttention - vLLM

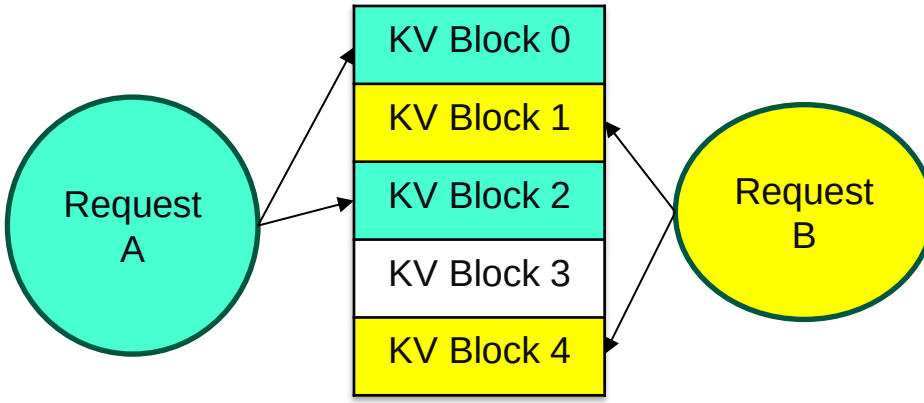
vLLM: Efficient memory management for LLM inference

Inspired by **virtual memory** and **paging**

Memory management in OS



Memory management in vLLM



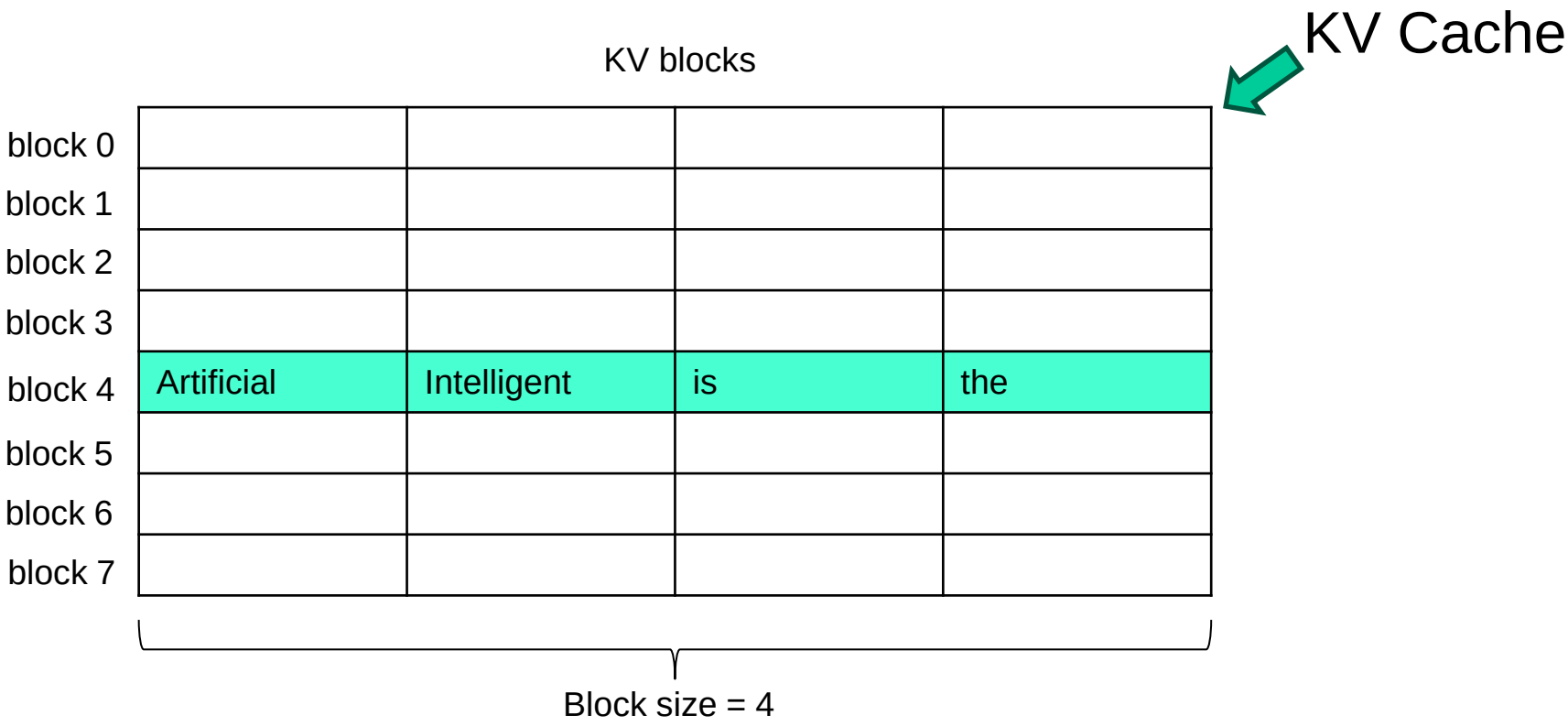
PagedAttention - vLLM

1. PagedAttention Algorithm
2. KV Cache Manager
3. Decoding with PagedAttention and vLLM
4. Application to Other Decoding Scenarios
5. Scheduling and Preemption
6. Distributed Excution
7. Implementation
8. Evaluation
9. Ablation Study

PagedAttention - vLLM

KV block

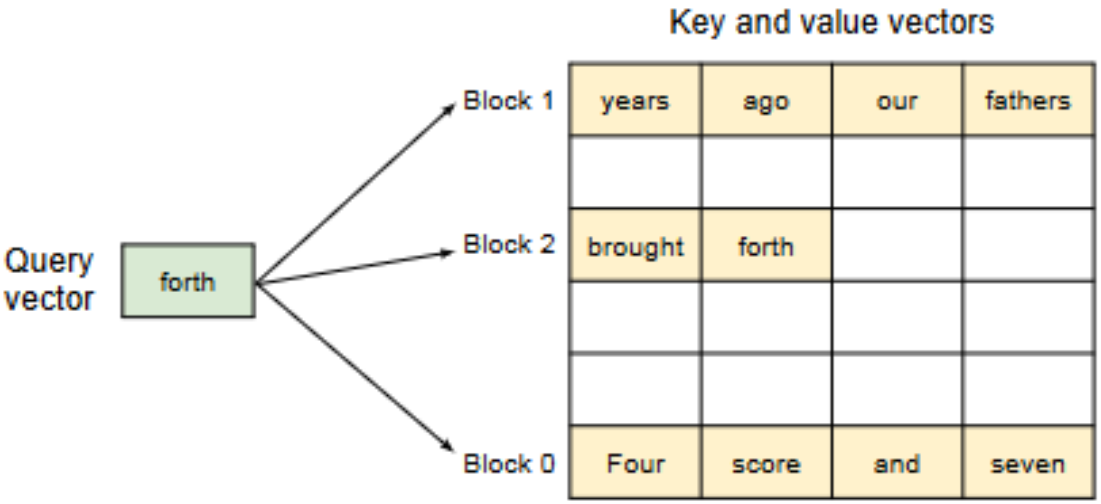
- A **fixed-size** contiguous chunk of memory that can store KV cache **from left to right**



PagedAttention - vLLM

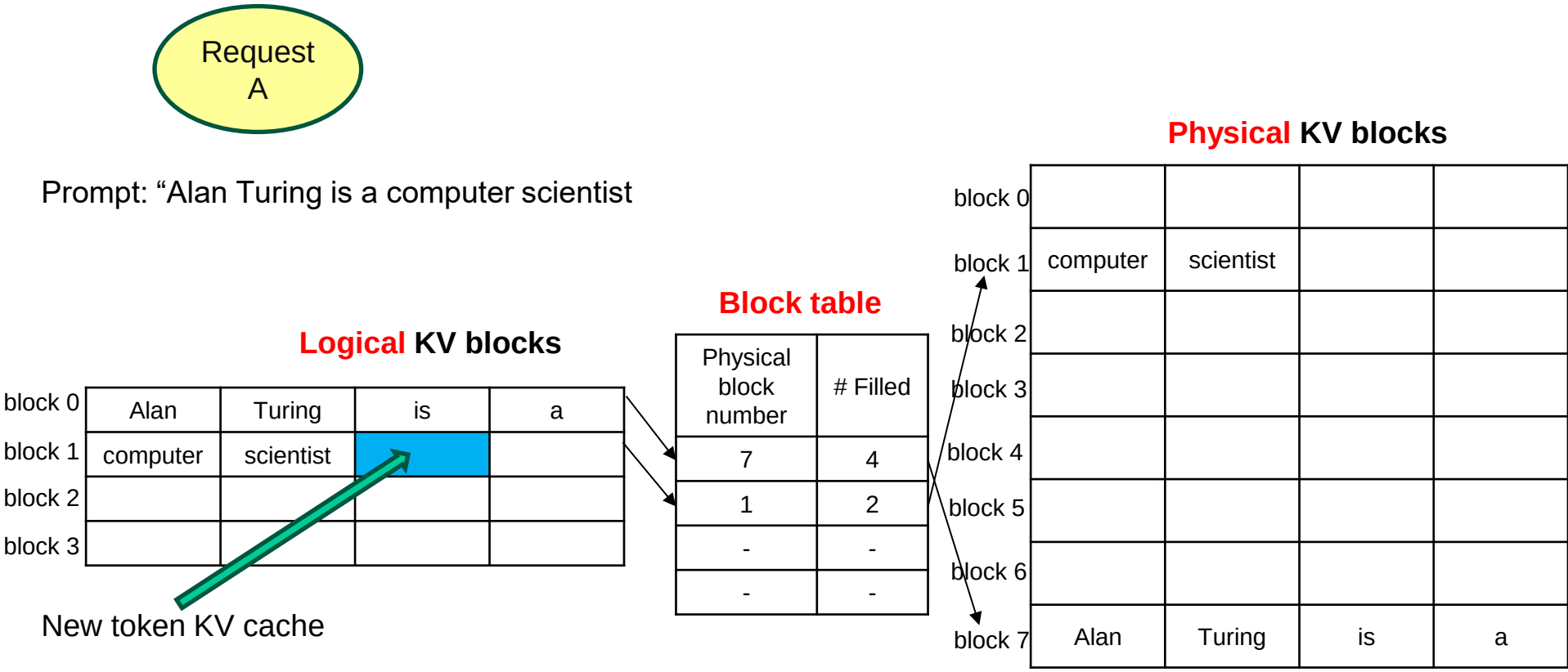
PagedAttention

- An algorithm that allows for storing continuous key and values in non-contiguous memory space



PagedAttention - vLLM

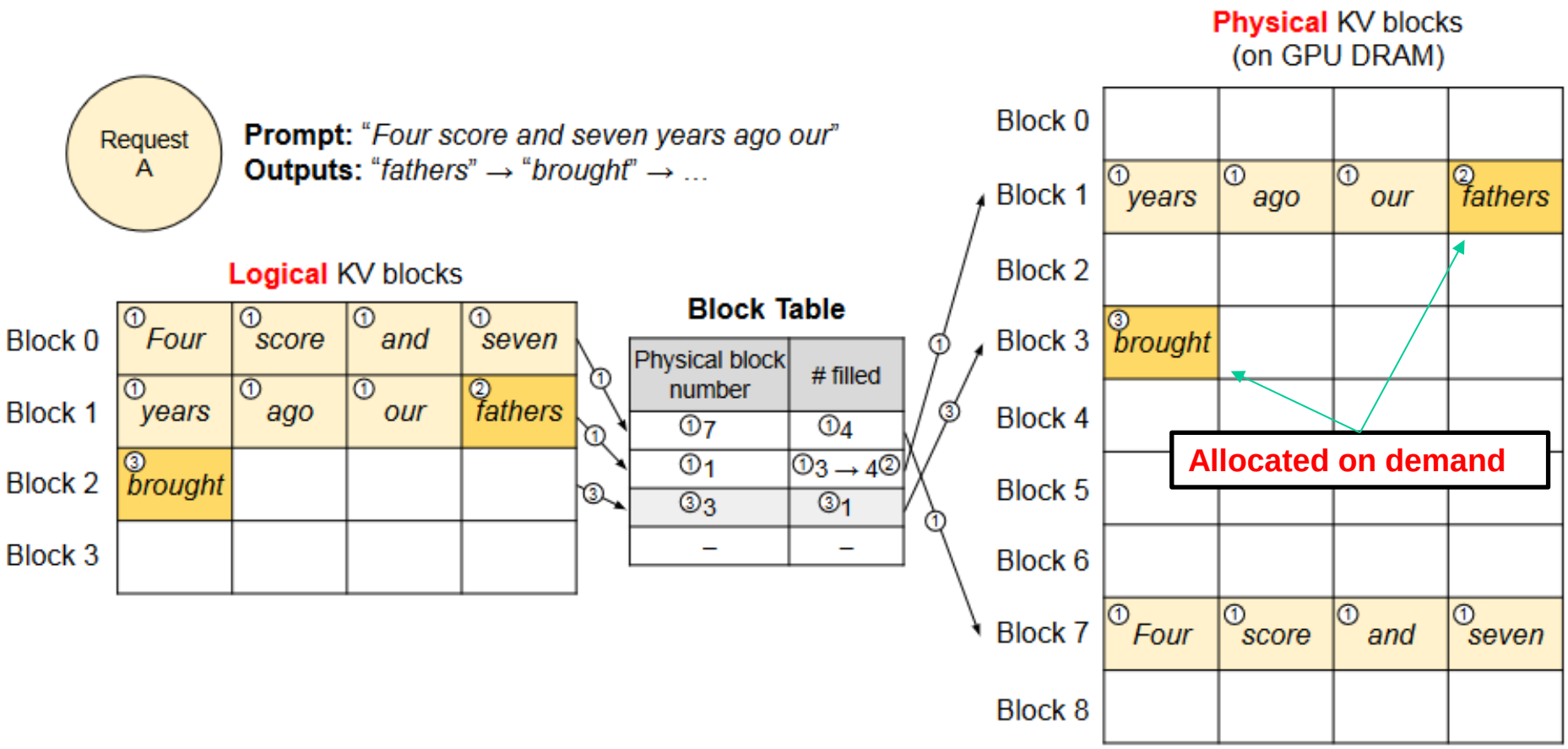
Logical & Physical KV blocks



- A block engine allocates a contiguous chunk of GPU DRAM then divides it into physical KV blocks

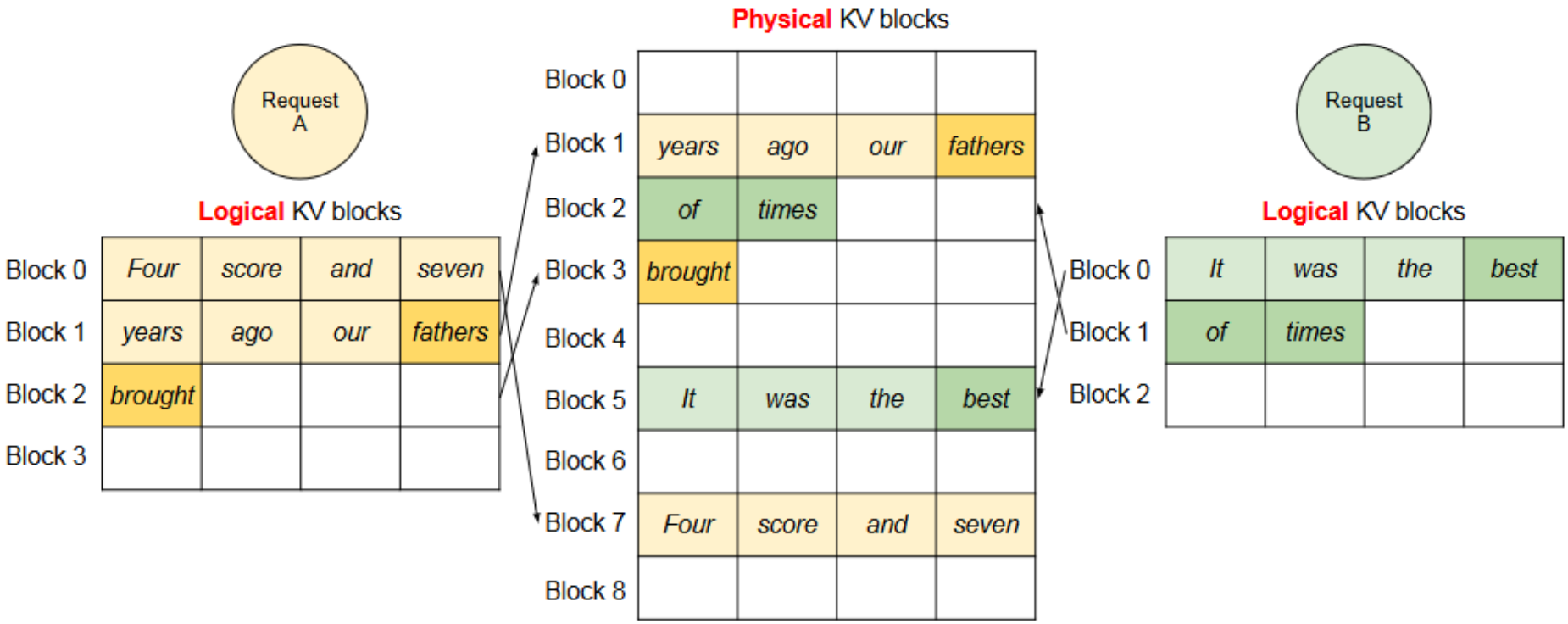
PagedAttention - vLLM

Decoding with PagedAttention and vLLM



PagedAttention - vLLM

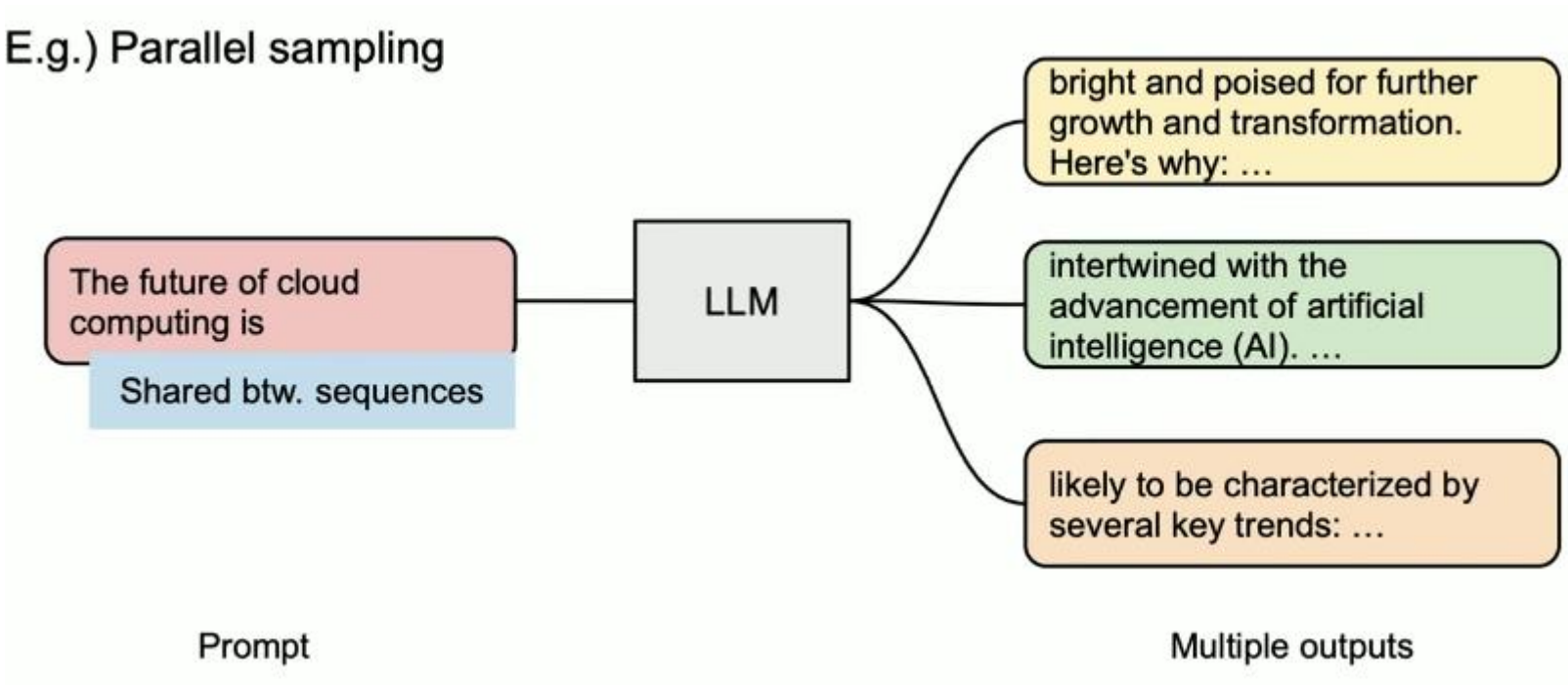
Decoding with PagedAttention and vLLM



PagedAttention - vLLM

Application to Other Decoding Scenarios

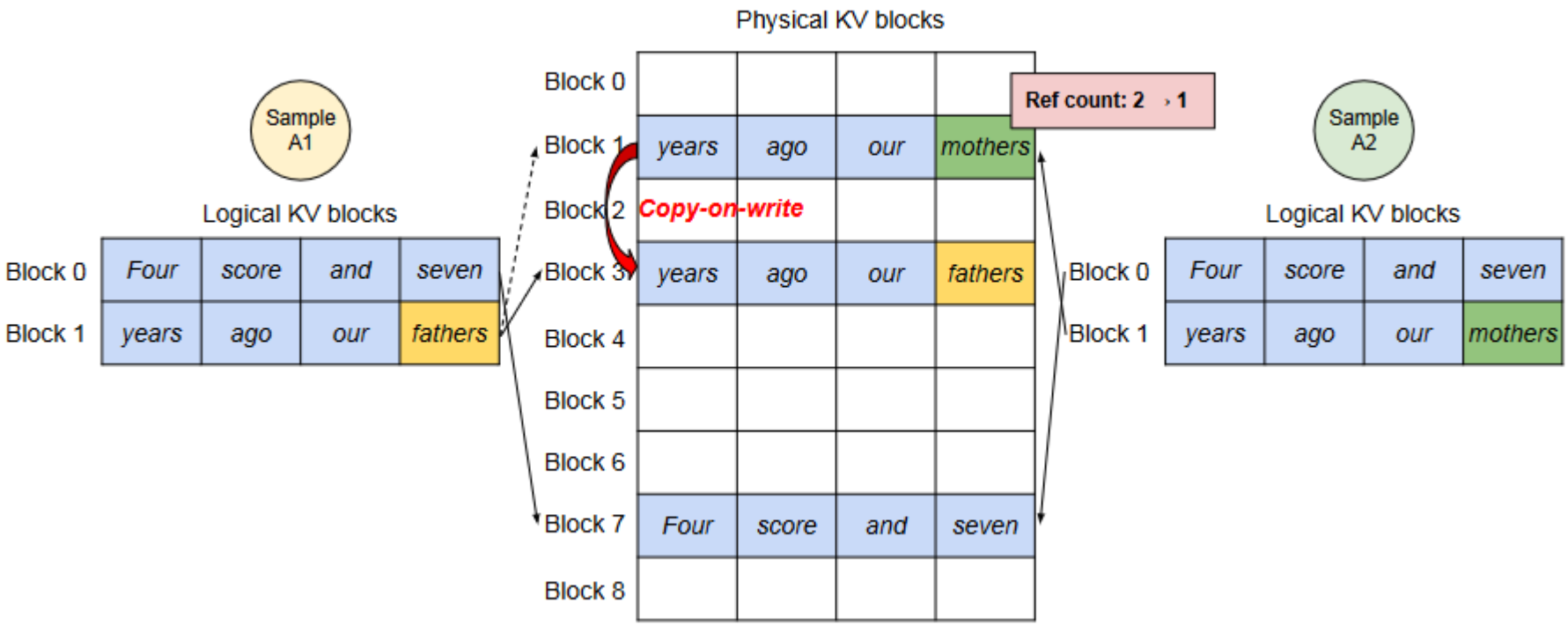
- Parallel sampling: Codex, Copilot



PagedAttention - vLLM

Application to Other Decoding Scenarios

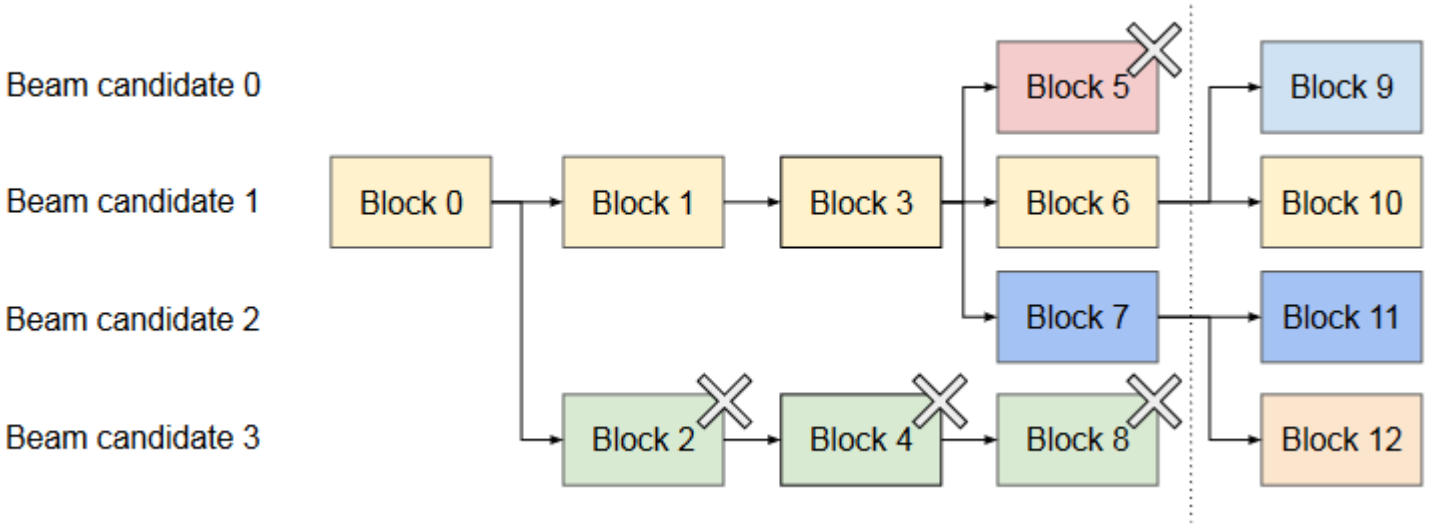
- Parallel sampling: Codex, Copilot



PagedAttention - vLLM

Application to Other Decoding Scenarios

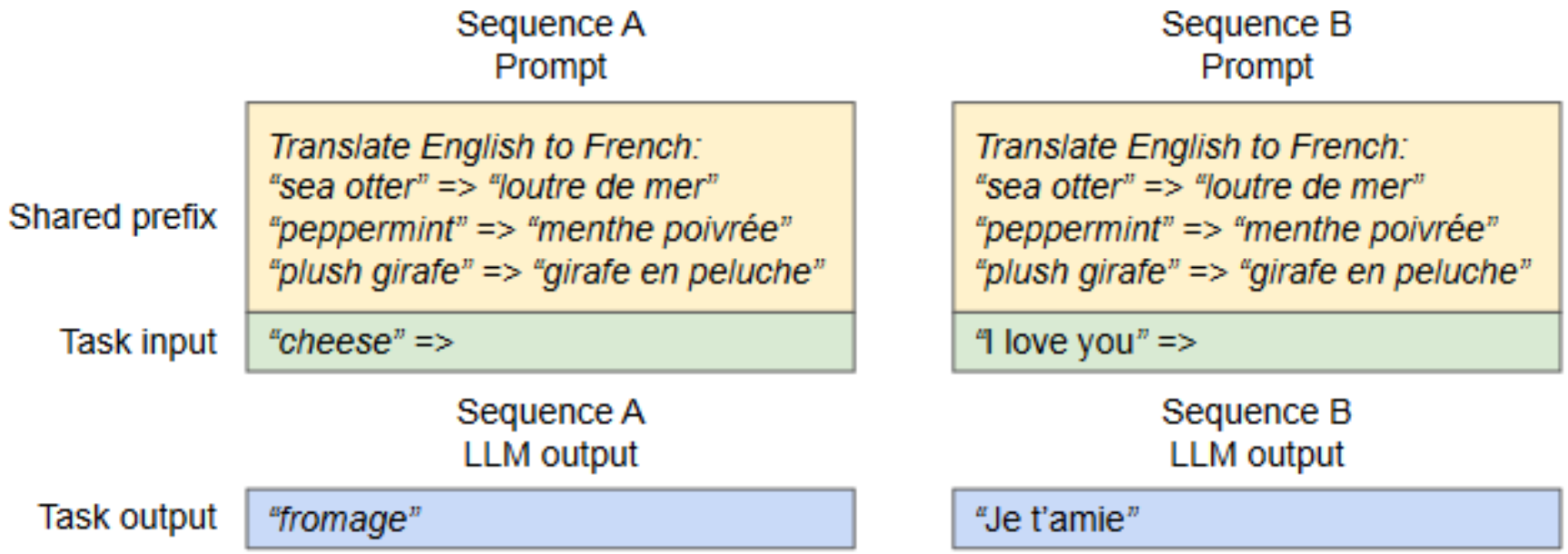
- Beam Search



PagedAttention - vLLM

Application to Other Decoding Scenarios

- Shared prefix



PagedAttention - vLLM

Scheduling and Preemption

- First-come-first-serve ~ lastest requests are preempted first
- When OOM:
  - All-or-nothing: all blocks of a sequence, a sequence group (beam candidates in one beam search request)
  - Recover evicted block:
    - ✓ Swapping (with CPU RAM)
      - Once preemption happened → stop accepting new accept until all preempted sequences are completed
      - Swap space bounded by GPU memory allocate for KV Cache
    - ✓ Re-computation: recompute the KV cache when the preempted sequences a re-scheduled

PagedAttention - vLLM

Distributed Execution (Megatron-LM, model parallel)

- LLMs parameter size exceeding the capacity of a single GPU
- vLLM features a single KV cache manager winthin the centralized schedular → different GPU works share the manager

  → Don’t need synchronize on memory management

PagedAttention - vLLM

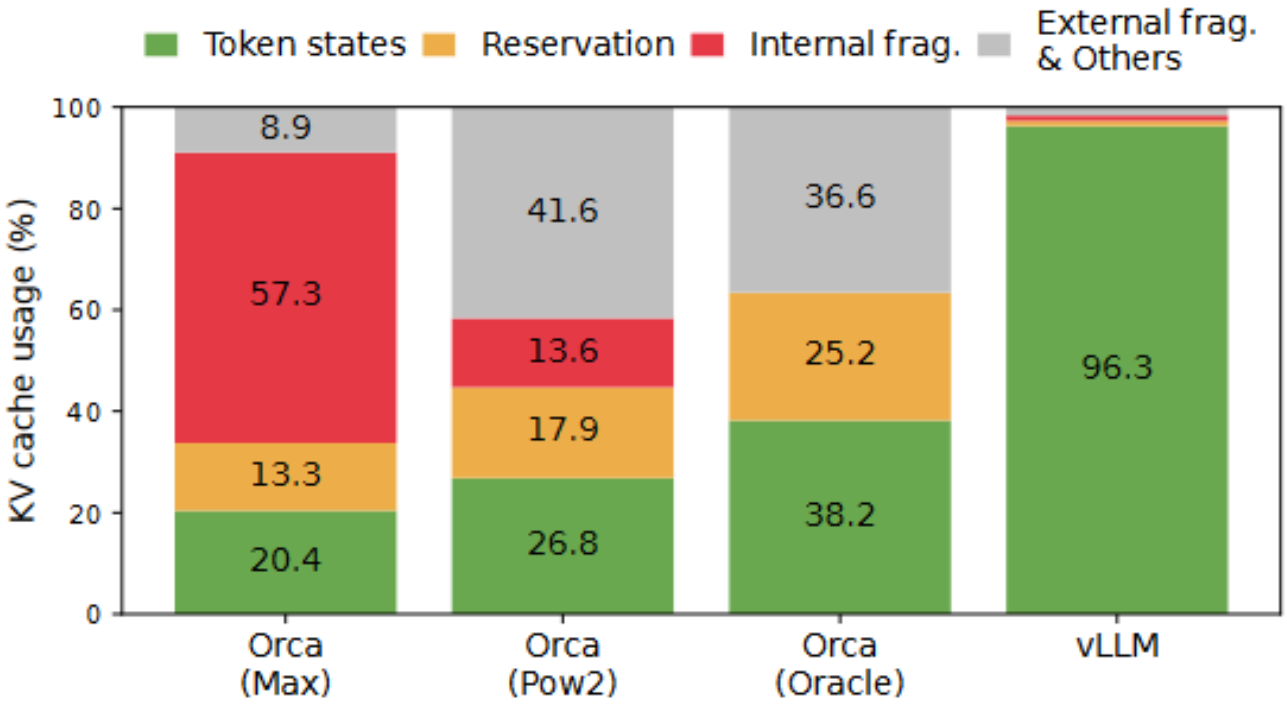
Implementation

- 8.5k LoC python (scheduler+block manager), 2K LoC C++/CUDA (PagedAttention)
- Kernel-level optimization:
  - Fused reshape and block: single kernel
  - Fuse block read and attention: adapt the attention kernel in FasterTransformer
  - Fuse block copy: batch the copy operations
- Decoding algorithms: fork, append, free

PagedAttention - vLLM

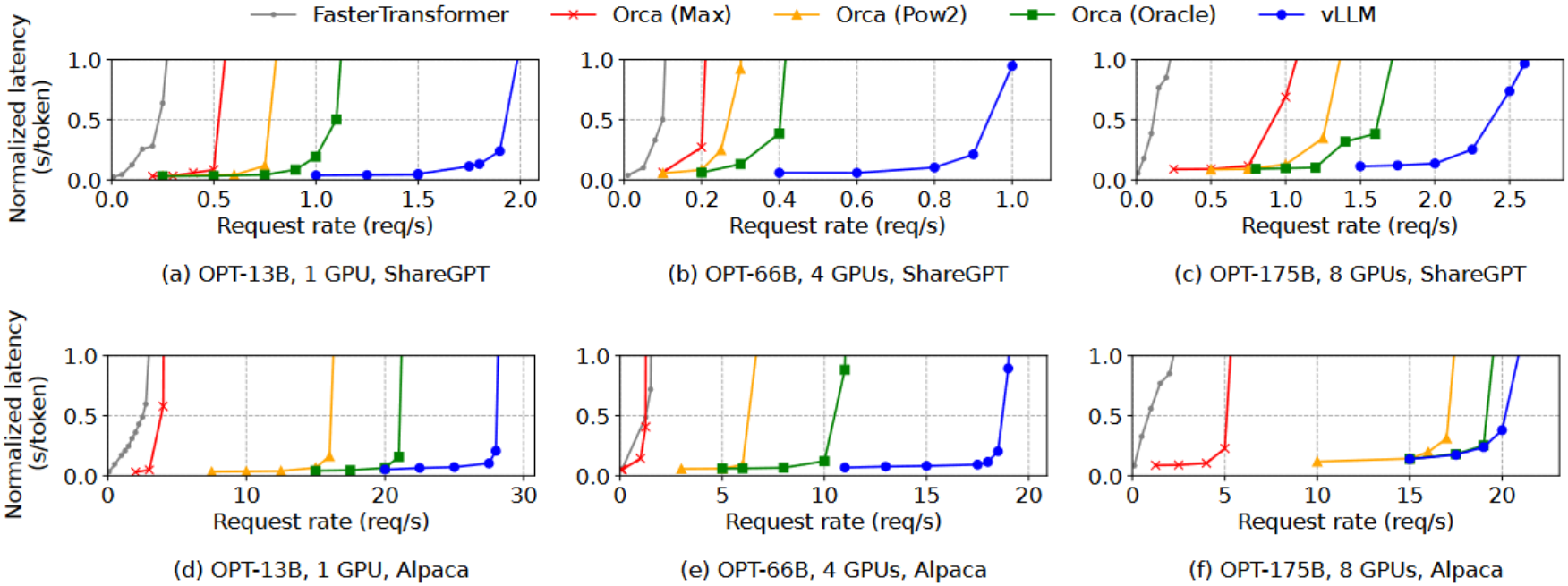
Evaluation

- Compare with SOTA ORCA (self implementation because of not public source – buddy allocator)



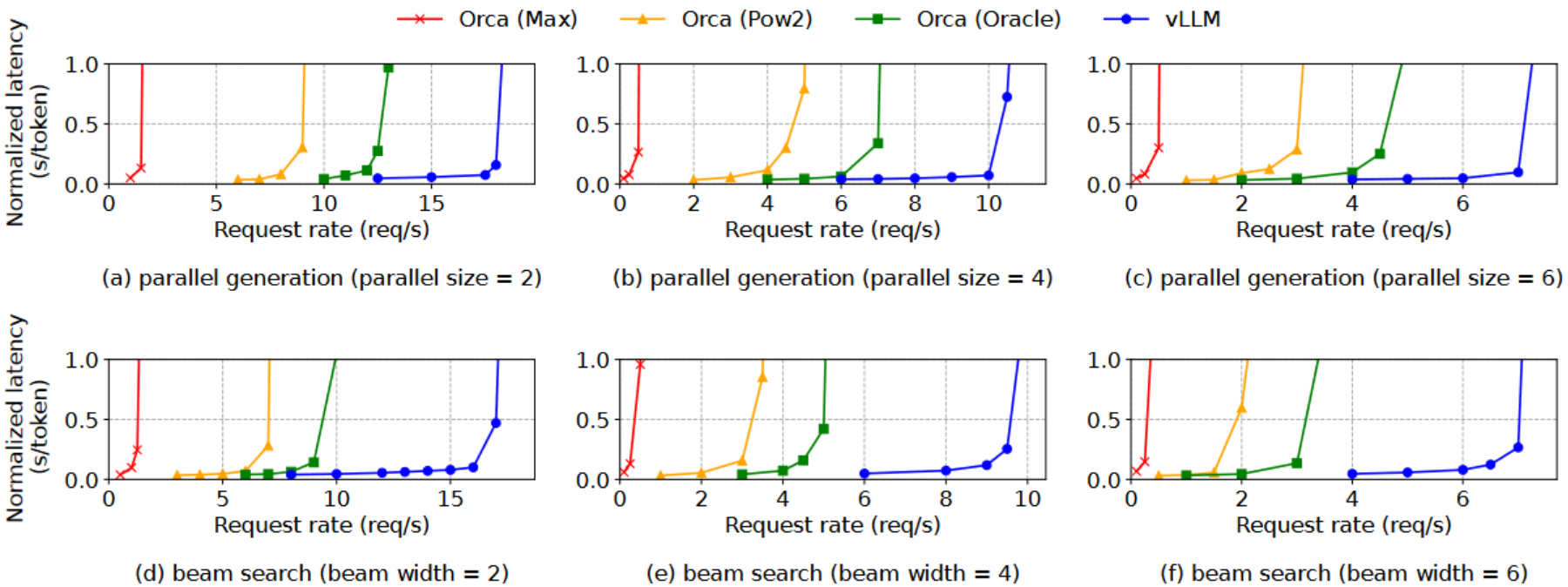
PagedAttention - vLLM

Evaluation



PagedAttention - vLLM

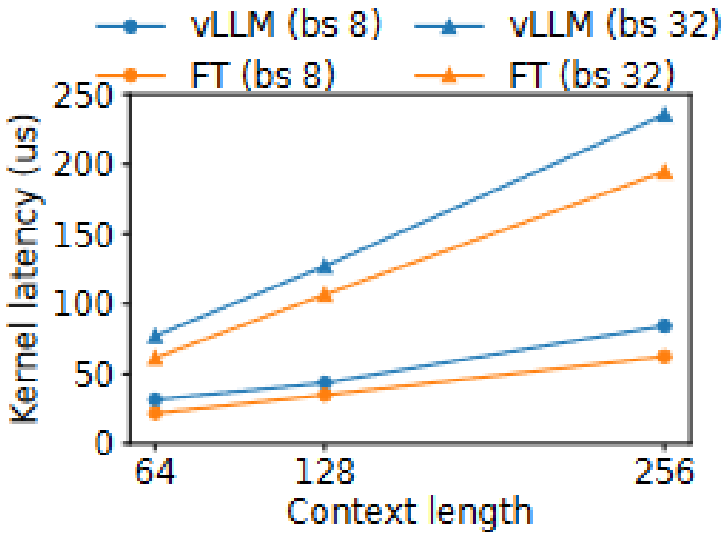
Evaluation



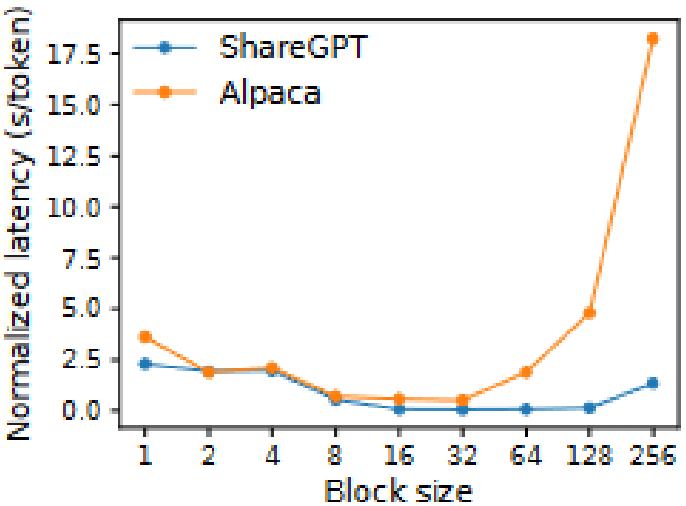
PagedAttention - vLLM

Ablation Studies

- Kernel Microbenchmark : 20-26% higher in attention kernel latency



(a) Latency of attention kernels.

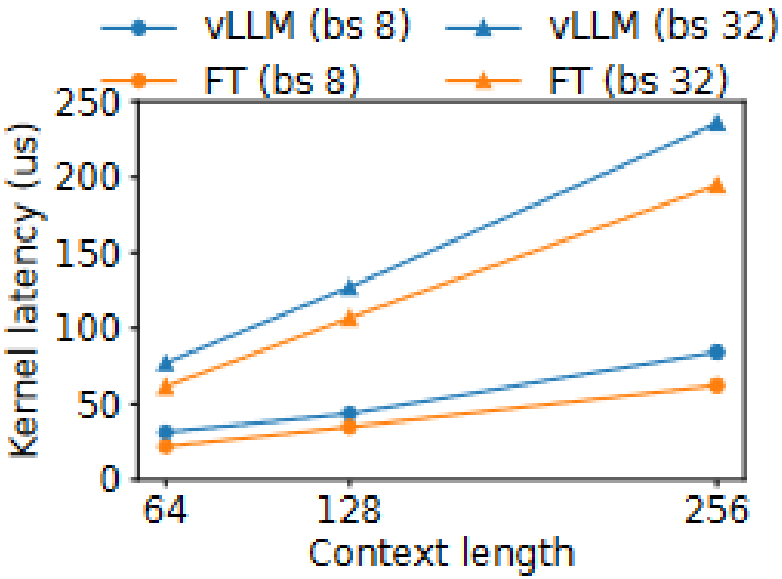


(b) End-to-end latency with different block sizes.

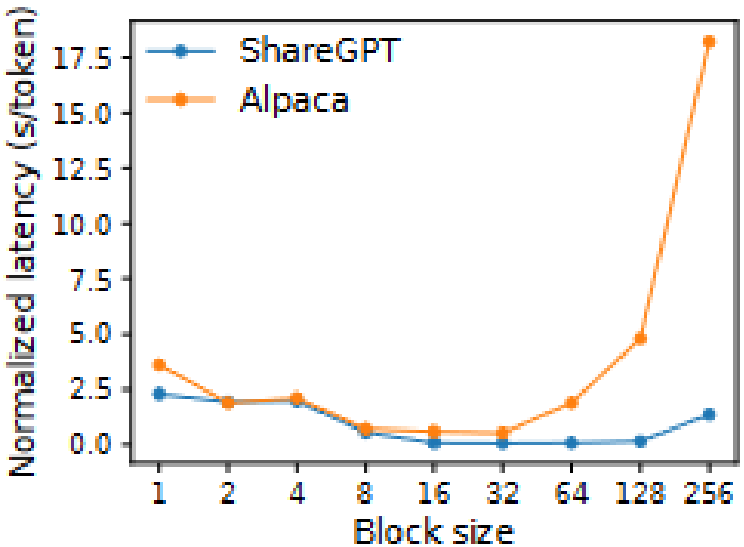
PagedAttention - vLLM

Ablation Studies

- Block size: default 16



(a) Latency of attention kernels.

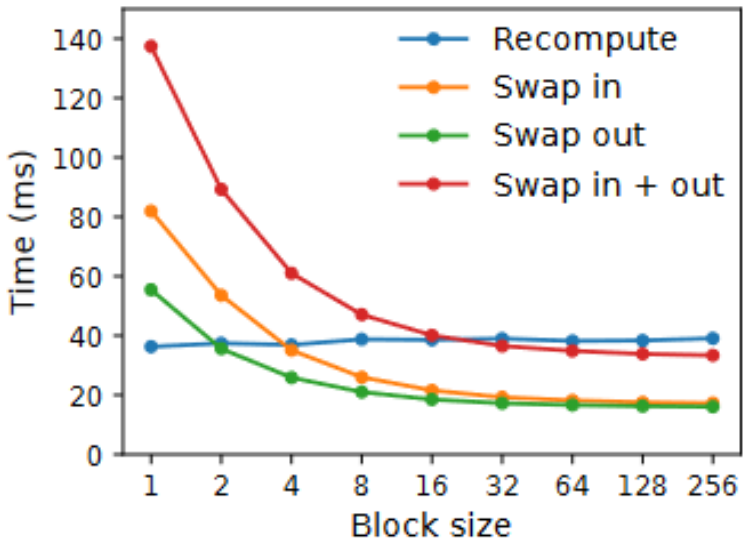


(b) End-to-end latency with different block sizes.

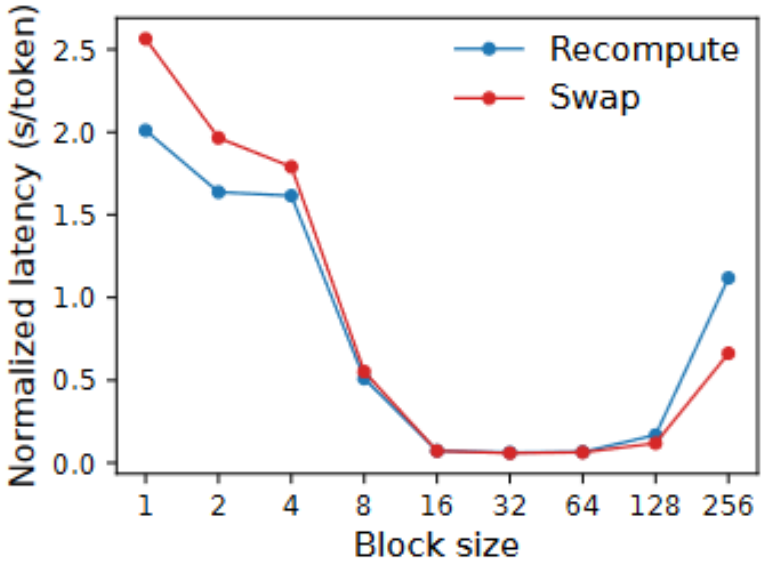
PagedAttention - vLLM

Ablation Studies

- Comparing Re-computation and Swapping



(a) Microbenchmark



(b) End-to-end performance

PagedAttention - vLLM

1. Problems with Large Language Model (LLM) Inference System
2. PagedAttention - vLLM
3. **Related works**

Related Work

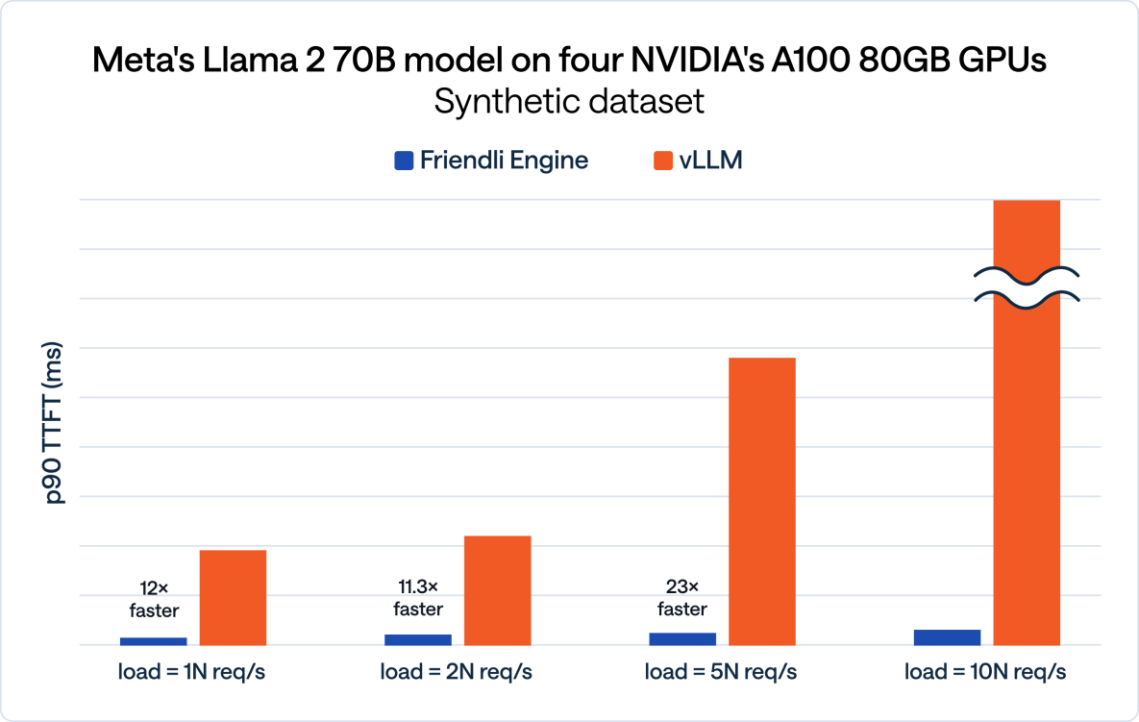
- **General model serving systems:**
  - Clipper, TensorFlow Serving, Nexus, InferLine, Clockwork: Batching, Caching, Placement, Scheduling
  - DVABatch: multi-entry muti-eixt batching
  - REEF, Shepherd: preemption
  - AlphaServe: model parallelism
- **Specialize serving systems for transformer:**
  - Utilize GPU kernel optimizations
  - Advanced batching mechanisms
  - Model parallelism
  - Parameter sharing

Related Work

- **Memory Optimizations**
  - Swapping
  - Re-computation
  - Combine Swapping and Re-computation
  - FlexGen: swap weights and tokens for LLM inference(offline)
  - FlashAttention: tiling and kernel optimizations in attention computation

Appendix

- March 2nd 2024: vLLM release v0.3.3
- April 2nd 2024: third vLLM meetup
- February 7th 2024, Friendli Engine (ORCA) show that their engine is better then vLLM



Appendix

❖ **vLLM v0.2 (January 2024)**

3 Months Ago -> Today

First-Priority Features

- ✅ Latency optimization
  - ✅ Lightweight runtime
  - 🔨 Multi-process architecture
  - 🔨 Speculative decoding
- ✅ Better quantization support
- ✅ CUDA-level optimizations

Second-Priority Features

- ✅ (Efficient) LoRA support
- ✅ AMD support
- ✅ Mixture-of-experts
- 🔨 Pipeline parallelism

+ Distributed Runtime Optimization
+ Prefix Caching
+ CI/CD